

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

RDBMS (SQL administracija & CASE alati)

BIOSKOP

(Seminarski rad)

Predmetni nastavnik

Doc.dr Dražen Marinković

Asistent

Pero Ranilović

Student: Nikolina Bekić

Br. Indeksa: 21-22-FIT-IIT1-PRO-240

Banja Luka, avgust 2026

SADRŽAJ

UVOD	1
1. ZAHTIJEVI ZA BAZU PODATAKA	2
2. KREIRANJE BAZE PODATAKA	4
3. KREIRANJE TABELA	6
3.1. Tabela „Zanrovi“	6
3.2. Tabela „Kategorije“	6
3.3. Tabela „Filmovi“	7
3.4. Tabela „Sale“	8
3.5. Tabela „Sjedista“	9
3.6. Tabela „Projekcije“	9
3.7. Tabela „Kupci“	11
3.8. Tabela „Rezervacije“	11
3.9. Tabela „RezervisanaSjedista“	12
3.10. Tabela „Placanja“	13
3.11. Tabela „Cjenovnik“	14
3.12. Tabela „Zaposleni“	15
3.13. Tabela „Smjene“	15
4. KONTROLA I INTEGRITET PODATAKA	16
4.1. Integritet na nivou entiteta	16
4.2. Integritet na nivou domena	16
4.3. Ograničenja za tabelu Kupci	16
4.4. Ograničenja za tabelu Placanja	17
4.5. Ograničenja za tabelu Rezervacije	17
4.6. Ograničenja za tabelu RezervisanaSjedista	17
4.7. Ograničenja za tabelu Cjenovnik	18
4.8. Ograničenja za tabelu Smjene	18
4.9. Referencijalni integritet u bazi	18
5. KREIRANJE INDEKSA ZA OPTIMIZACIJU PERFORMANSI	20
5.1. Indeks na koloni Naziv u tabeli Filmovi	20
5.2. Indeks na koloni DatumVrijeme u tabeli Projekcije	20
5.3. Kompozitni indeks na kolonama FilmID i StatusProjekcije u tabeli Projekcije	20
5.4. Indeks na kolonama SalaID i RedOznaka u tabeli Sjedista	20
5.5. Indeks na koloni KupacID u tabeli Rezervacije	21

5.6. Kompozitni indeks na kolonama ProjekcijaID i StatusRezervacije u tabeli Rezervacije .	21
5.7. Indeks na koloni DatumPlacanja u tabeli Placanja	21
5.8. Filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja	21
6. UNOS PODATAKA U BAZU PODATAKA	22
6.1. Unos podataka u tabelu Završi	22
6.2. Unos podataka u tabelu Kategorije	22
6.3. Unos podataka u tabelu Sale	23
6.4. Unos podataka u tabelu Filmovi	23
6.5. Unos podataka u tabelu Sjedista	24
6.6. Unos podataka u tabelu Kupci	25
6.7. Unos podataka u tabelu Zaposleni	25
6.8. Unos podataka u tabelu Smjene	26
6.9. Unos podataka u tabelu Cjenovnik	26
6.10. Unos podataka u tabelu Projekcije	27
6.11. Unos podataka u tabelu Rezervacije	27
6.12. Unos podataka u tabelu RezervisanaSjedista	28
6.13. Unos podataka u tabelu Placanja	28
7. KREIRANJE POGLEDA ZA JEDNOSTAVNIJI PRISTUP PODACIMA	29
7.1. Pogled vw_RasporedProjekcija	29
7.2. Pogled vw_RezervacijeDetalji	30
7.3. Pogled vw_PrihodPoFilmu	31
8. KREIRANJE KORISNIČKIH FUNKCIJA ZA PONOVO KORIŠTENJE KODA	32
8.1. Funkcija fn_PopustKupca	32
8.2. Funkcija fn_SlobodnaSjedista	33
8.3. Funkcija fn_PrihodProjekcije	34
9. KREIRANJE USKLADIŠTENIH PROCEDURA ZA AUTOMATIZACIJU	35
9.1. Procedura sp_KreirajRezervaciju	35
9.2. Procedura sp_OtkaziRezervaciju	36
9.3. Procedura sp_ProjekcijeZaPeriod	36
9.4. Procedura sp_PrihodPeriod	37
10. KREIRANJE TRIGERA ZA AUTOMATSKO IZVRŠAVANJE RADNJI	38
10.1. Tabela za praćenje promjena rezervacija	38
10.2. Trigger trg_Rezervacije_Dnevnik	38
10.3. Tabela za praćenje promjena cijena projekcija	39

10.4. Triger trg_Projekcije_Cijena	39
10.5. Triger trg_RS_ProvjeraSale.....	40
11. PRIMJERI DML NAREDBI ZA MANIPULACIJU PODACIMA.....	41
11.1. SELECT naredba za dohvaćanje podataka.....	41
11.2. INSERT naredba za unos novih podataka	41
11.3. UPDATE naredba za izmjenu postojećih podataka.....	42
11.4. DELETE naredba za brisanje podataka	42
12. PRIMJERI DDL NAREDBI ZA DEFINISANJE STRUKTURE.....	43
12.1. ALTER naredba za izmjenu strukture	43
12.2. DROP naredba za brisanje objekata	43
12.3. CREATE naredba za kreiranje objekata	43
12.4. TRUNCATE naredba za brzo brisanje svih podataka	43
13. PRIMJERI KORIŠTENJA RAZLIČITIH TIPOVA FUNKCIJA	45
13.1. Datumske funkcije	45
13.2. String funkcije.....	45
13.3. Agregatne funkcije.....	45
13.4. Numeričke funkcije.....	46
14. PRIMJERI JOIN UPITA ZA KOMBINOVANJE PODATAKA IZ VIŠE TABELA.....	47
14.1. INNER JOIN.....	47
14.2. LEFT JOIN	47
14.3. RIGHT JOIN.....	47
14.4. FULL OUTER JOIN.....	48
14.5. CROSS JOIN	48
14.6. SELF JOIN.....	48
15. PRIMJERI PODUPITA I SKUPOVNIH OPERATORA.....	49
15.1. Podupiti.....	49
15.2. Skupovni operatori.....	49
16. KORISNICI, ULOGE I BEZBJEDNOST	51
16.1. Kreiranje uloga.....	51
16.2. Dodjela prava ulogama	51
16.3. Kreiranje korisnika.....	51
17. ETL PROCESI.....	53
17.1. Tabela StageFilmovi	53
17.2. Procedura sp_UveziFilmove.....	53

17.3. Tabela DnevniPrihod	55
17.4. Procedura sp_UcitajDnevniPrihod.....	55
18. KREIRANJE REZERVNE KOPIJE BAZE PODATAKA	56
18.1. Kreiranje potpunog backup-a.....	56
18.2. Diferencijalni backup.....	56
18.3. Backup loga transakcija	56
18.4. Copy-only backup.....	57
18.5. Ostale vrste backup-a.....	57
18.6. Informacije o backup-u	58
18.7. Restauracija baze iz backup-a.....	58
19. PUBLIKACIJA I TRANSAKCIJSKA REPLIKACIJA.....	59
19.1. Konfiguracija Distributora	59
19.2. Kreiranje publikacije.....	59
19.3. Dodavanje artikala	60
19.4. Kreiranje pretplate	60
20. ER DIJAGRAM I OPIS RELACIJA	61
20.1. Entiteti i njihovi atributi.....	61
20.2. Veze između entiteta.....	62
20.3. Grafički prikaz ER dijagrama	62
ZAKLJUČAK	64
POPIS SLIKA.....	65

UVOD

Savremeno poslovanje bioskopa zahtijeva vođenje velikog broja podataka koji se odnose na filmove, projekcije, sale, kupce, rezervacije i plaćanja. Kako broj projekcija i korisnika raste, ručno vođenje evidencije ili korištenje jednostavnih tabela postaje sve teže i povećava se mogućnost nastanka grešaka. Zbog toga je potrebno imati sistem u kojem su svi podaci organizovani na jednom mjestu i međusobno povezani.

Baza podataka predstavlja jedan od osnovnih dijelova takvog sistema. Njena uloga je da omogući čuvanje i povezivanje podataka o filmovima, salama, sjedištima, terminima projekcija, kupcima i njihovim rezervacijama. Na taj način se podaci mogu lakše pretraživati, mijenjati i koristiti u različitim dijelovima sistema.

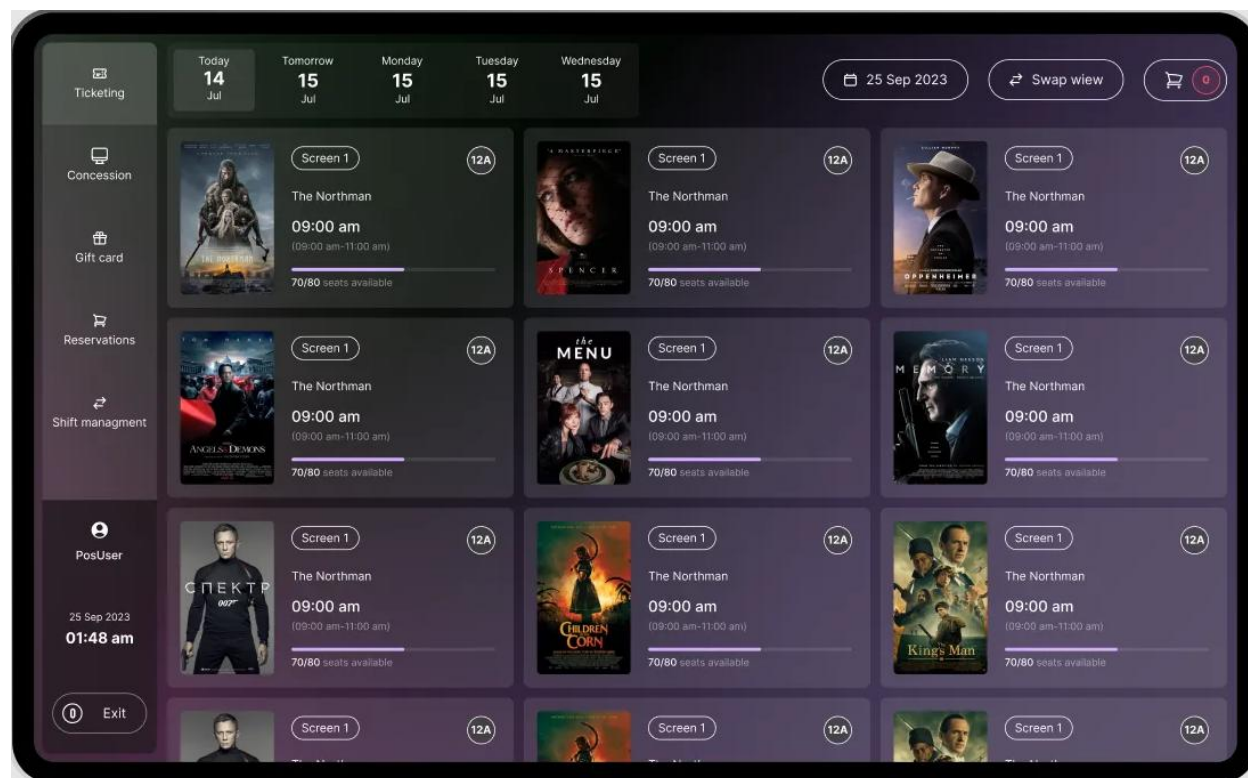
U ovom radu prikazano je projektovanje i implementacija baze podataka Bioskop. Posebna pažnja posvećena je organizaciji tabela i njihovim međusobnim vezama, kao i korištenju primarnih i stranih ključeva, ograničenja, indeksa, procedura, funkcija i trigera. Pored osnovnog modela baze, obrađeni su i pogledi, ETL procesi, korisničke uloge, backup i replikacija.

Cilj rada je da se napravi funkcionalna i logički organizovana baza koja može podržati osnovne potrebe jednog bioskopa, od evidencije filmova i projekcija do rezervacije sjedišta i obrade plaćanja. Ovako projektovana baza može poslužiti kao osnova za izradu kompletnog informacionog sistema za upravljanje bioskopskim poslovanjem.

1. ZAHTJEVI ZA BAZU PODATAKA

Prije početka projektovanja baze podataka, neophodno je izvršiti detaljnu analizu zahtjeva koja definiše šta sistem treba da podržava. U slučaju bioskopa, osnovni zahtjevi obuhvataju evidentiranje svih filmova koji se prikazuju, sa pripadajućim žanrovima i dobnim kategorijama koje određuju ko može prisustvovati projekciji. Svaki film ima svoj naziv, originalni naziv, trajanje, godinu proizvodnje, datum premijere i prosječnu ocjenu, te oznaku da li je trenutno aktivan.

Bioskop raspolaže sa više sala koje imaju različit kapacitet i tip, kao što su standardne sale, 3D sale, IMAX sale i VIP salon. Svaka sala sadrži određeni broj sjedišta koja su organizovana po redovima i brojevima, pri čemu sjedišta mogu biti standardna, VIP ili pristupačna za osobe sa posebnim potrebama. Ova struktura omogućava precizno upravljanje kapacitetom i prodajom karata



Slika 1 - Primjer softverskog rješenja za upravljanje bioskopom kompanije CINESync

Projekcije su centralni entitet sistema jer povezuju filmove sa salama u određenom vremenskom terminu. Svaka projekcija ima svoj datum i vrijeme početka, jezik na kojem se film prikazuje, format projekcije i osnovnu cijenu. Status projekcije može biti planirana, u toku, završena ili otkazana, što omogućava praćenje realizacije rasporeda.

Kupci su registrovani korisnici koji vrše rezervacije. Svaki kupac ima svoje osnovne podatke, email adresu, telefon, datum registracije i broj posjeta. Rezervacije se kreiraju za određenu projekciju i obuhvataju jedno ili više sjedišta. Svaka rezervacija ima svoj status, koji može biti na čekanju, potvrđena, otkazana ili iskorištena. Plaćanja se vrše za rezervacije i mogu se obaviti na različite načine, uključujući gotovinu, karticu, online plaćanje ili vaucher.

Sistem također vodi evidenciju o zaposlenima i njihovim smjenama, što je važno za organizaciju rada u bioskopu. Cjenovnik definiše cijene karata u zavisnosti od tipa sale i dana u sedmici, što omogućava primjenu različitih cjenovnih politika.

Izvještaji koji su potrebni obuhvataju pregled rasporeda projekcija, detalje rezervacija, prihod po filmu, slobodna sjedišta za određenu projekciju, te dnevni prihod po filmovima. Ovi izvještaji su ključni za operativno upravljanje i strateško planiranje.

2. KREIRANJE BAZE PODATAKA

Prvi korak u izradi projekta jeste kreiranje baze podataka pod nazivom Bioskop. Time se uspostavlja osnovna struktura sistema u kojoj će se čuvati i organizovati svi podaci vezani za bioskopsko poslovanje. U okviru sistema SQL Server, baza podataka predstavlja logičku cjelinu koja objedinjuje sve tabele, veze i ostale objekte potrebne za rad aplikacije.

```
CREATE DATABASE Bioskop;  
GO
```

Slika 2 - Kreiranje baze podataka

Nakon kreiranja baze podataka, aktivira se za dalji rad korištenjem naredbe USE, kojom se definiše da će se sve naredne operacije izvršavati nad bazom Bioskop. Aktivna baza može se izabrati i putem grafičkog korisničkog interfejsa u okviru SQL Server Management Studio okruženja.

```
USE Bioskop;  
GO
```

Slika 3 - Aktiviranje baze podataka

Organizacija objekata unutar baze podataka postignuta je korištenjem shema. Sheme predstavljaju logičke kontejnere koji omogućavaju grupisanje objekata prema njihovoj namjeni. U okviru ove baze definisano je nekoliko shema. Shema sif sadrži šifarnike, odnosno tabele sa osnovnim podacima poput žanrova i kategorija. Shema kino obuhvata tabele vezane za filmski sadržaj i fizičku strukturu bioskopa, uključujući filmove, sale, sjedišta, projekcije, zaposlene i smjene. Shema prodaja sadrži tabele koje se odnose na prodajne procese, kao što su kupci, rezervacije, rezervisana sjedišta, plaćanja i cjenovnik. Shema izvjestaji obuhvata poglede i tabele za izvještavanje. Shema audit sadrži tabele za praćenje promjena, dok shema etl sadrži tabele i procedure za ETL procese.

```
CREATE SCHEMA sif AUTHORIZATION dbo;  
GO  
CREATE SCHEMA kino AUTHORIZATION dbo;  
GO  
CREATE SCHEMA prodaja AUTHORIZATION dbo;  
GO  
CREATE SCHEMA izvjestaji AUTHORIZATION dbo;  
GO  
CREATE SCHEMA audit AUTHORIZATION dbo;  
GO  
CREATE SCHEMA etl AUTHORIZATION dbo;  
GO
```

Slika 4 - Kreiranje shema u bazi podataka

Ovakva podjela poboljšava preglednost baze podataka i olakšava upravljanje objektima. Također, omogućava precizniju dodjelu prava pristupa na nivou poslovnih cjelina, što je važno za bezbjednost sistema. Na primjer, korisnicima koji rade na prodaji može se dati pristup samo shemi prodaja, dok analitičari mogu imati pristup shemi izvjestaji.

3. KREIRANJE TABELA

Nakon kreiranja baze podataka i shema, slijedi definisanje tabela koje će čuvati podatke o glavnim entitetima sistema. Svaka tabela sadrži kolone sa odgovarajućim tipovima podataka, a za svaku je definisan primarni ključ radi jedinstvene identifikacije zapisa. Veze između tabela ostvarene su pomoću stranih ključeva.

3.1. Tabela „Zanrovi“

Tabela Zanrovi predstavlja šifarnik žanrova filmova. Ova tabela je ključna za kategorizaciju filmova i omogućava pretragu po žanru. Prva kolona u ovoj tabeli je ZanrID, koja je definisana kao IDENTITY kolona. Identity svojstvo znači da će SQL Server automatski generisati jedinstvenu numeričku vrijednost za svaki novi red, počevši od 1 i povećavajući se za 1. Ova kolona je postavljena kao primarni ključ tabele.

```
CREATE TABLE sif.Zanrovi (  
    ZanrID INT IDENTITY PRIMARY KEY,  
    Naziv NVARCHAR(50) NOT NULL UNIQUE,  
    Opis NVARCHAR(300)  
);
```

Slika 5 - Query za kreiranje tabele Zanrovi

Kolona Naziv je definisana kao NVARCHAR(50) sa ograničenjem NOT NULL i UNIQUE, što znači da svaki žanr mora imati naziv i da naziv mora biti jedinstven. NVARCHAR tip podataka omogućava čuvanje Unicode znakova, što je važno za podršku različitih jezika i specijalnih karaktera. Kolona Opis je opcionalno polje koje može biti NULL.

3.2. Tabela „Kategorije“

Tabela Kategorije definiše starosne kategorije filmova. Svaka kategorija ima svoj naziv i minimalnu dob koja se zahtijeva za gledanje filmova iz te kategorije. KategorijaID je primarni

ključ sa identity svojstvom. Naziv je obavezan i jedinstven. Minimalna dob je definisana kao INTEGER sa CHECK ograničenjem koje osigurava da vrijednost bude između 0 i 21.

```
CREATE TABLE sif.Kategorije (  
    KategorijaID INT IDENTITY PRIMARY KEY,  
    Naziv NVARCHAR(50) NOT NULL UNIQUE,  
    MinimalnaDob INT NOT NULL  
        CHECK (MinimalnaDob BETWEEN 0 AND 21)  
);
```

Slika 6 - Query za kreiranje tabele Kategorije

3.3. Tabela „Filmovi“

Tabela Filmovi je centralni entitet za sve informacije o filmovima koji se prikazuju u bioskopu. FilmID je primarni ključ sa identity svojstvom. ZanrID i KategorijaID su strani ključevi koji povezuju film sa odgovarajućim šifarnicima. Ove kolone su obavezne jer svaki film mora pripadati nekom žanru i nekoj kategoriji.

```
CREATE TABLE kino.Filmovi (  
    FilmID INT IDENTITY PRIMARY KEY,  
    ZanrID INT NOT NULL,  
    KategorijaID INT NOT NULL,  
    Naziv NVARCHAR(150) NOT NULL,  
    OriginalniNaziv NVARCHAR(150),  
    TrajanjeMin INT NOT NULL,  
    Godina SMALLINT NOT NULL,  
    DatumPremijere DATE,  
    ProsjecnaOcjena DECIMAL(3,1),  
    Aktivan BIT NOT NULL DEFAULT 1,  
  
    CONSTRAINT FK_Filmovi_Zanrovi  
        FOREIGN KEY (ZanrID)  
        REFERENCES sif.Zanrovi(ZanrID),  
  
    CONSTRAINT FK_Filmovi_Kategorije  
        FOREIGN KEY (KategorijaID)  
        REFERENCES sif.Kategorije(KategorijaID),  
  
    CONSTRAINT UQ_Filmovi  
        UNIQUE (Naziv, Godina),  
  
    CONSTRAINT CHK_Filmovi_Trajanje  
        CHECK (TrajanjeMin BETWEEN 20 AND 400),  
  
    CONSTRAINT CHK_Filmovi_Godina  
        CHECK (Godina BETWEEN 1900 AND 2100),  
  
    CONSTRAINT CHK_Filmovi_Ocjena  
        CHECK (  
            ProsjecnaOcjena IS NULL  
            OR ProsjecnaOcjena BETWEEN 1 AND 10  
        )  
);
```

Slika 7 - Query za kreiranje tabele Filmovi

Naziv filma je obavezan i definisan je kao NVARCHAR(150). OriginalniNaziv je opciono polje. TrajanjeMin definiše dužinu filma u minutima i obavezno je polje sa CHECK ograničenjem koje osigurava da trajanje bude između 20 i 400 minuta, što pokriva sve uobičajene dužine filmova. Godina je obavezna i ograničena na raspon od 1900 do 2100. DatumPremijere je opcionalan. ProsjecnaOcjena je decimalno polje koje može biti NULL, a ako nije NULL, mora biti između 1 i 10. Aktivan je logičko polje sa podrazumijevanom vrijednošću 1, što znači da je film aktivan po defaultu.

Ograničenje UQ_Filmovi osigurava da kombinacija naziva i godine bude jedinstvena, što sprečava unos duplih filmova. Strani ključevi FK_Filmovi_Zanrovi i FK_Filmovi_Kategorije osiguravaju referencijalni integritet.

3.4. Tabela „Sale“

Tabela Sale čuva podatke o svim salama u bioskopu. SalaID je primarni ključ sa identity svojstvom. Naziv sale je obavezan i jedinstven. TipSale definiše vrstu sale i može imati vrijednosti Standard, 3D, IMAX ili VIP. Kapacitet je obavezan i mora biti veći od nule. DatumOtvaranja je opciono polje. Aktivna je logičko polje sa podrazumijevanom vrijednošću 1.

```
CREATE TABLE kino.Sale (
    SalaID INT IDENTITY PRIMARY KEY,
    Naziv NVARCHAR(50) NOT NULL UNIQUE,
    TipSale NVARCHAR(20) NOT NULL,
    Kapacitet INT NOT NULL,
    DatumOtvaranja DATE,
    Aktivna BIT NOT NULL DEFAULT 1,

    CONSTRAINT CHK_Sale_Tip
        CHECK (TipSale IN (N'Standard', N'3D', N'IMAX', N'VIP')),

    CONSTRAINT CHK_Sale_Kapacitet
        CHECK (Kapacitet > 0)
);
```

Slika 8 - Query za kreiranje tabele Sale

Ograničenje CHK_Sale_Tip definiše dozvoljene vrijednosti za tip sale, dok CHK_Sale_Kapacitet osigurava da kapacitet bude pozitivan broj.

3.5. Tabela „Sjedista“

Tabela Sjedista predstavlja fizički raspored sjedišta u svakoj sali. SjedisteID je primarni ključ sa identity svojstvom. SalaID je strani ključ koji povezuje sjedište sa odgovarajućom salom. RedOznaka i BrojSjedista zajedno sa SalaID čine jedinstvenu kombinaciju koja omogućava preciznu identifikaciju svakog sjedišta. TipSjedista definiše vrstu sjedišta i može imati vrijednosti Standard, VIP ili Pristupacno. Aktivno je logičko polje sa podrazumijevanom vrijednošću 1.

```
CREATE TABLE kino.Sjedista (  
    SjedisteID INT IDENTITY PRIMARY KEY,  
    SalaID INT NOT NULL,  
    RedOznaka NVARCHAR(5) NOT NULL,  
    BrojSjedista INT NOT NULL,  
    TipSjedista NVARCHAR(20) NOT NULL DEFAULT N'Standard',  
    Aktivno BIT NOT NULL DEFAULT 1,  
  
    CONSTRAINT FK_Sjedista_Sale  
        FOREIGN KEY (SalaID)  
        REFERENCES kino.Sale(SalaID),  
  
    CONSTRAINT UQ_Sjedista  
        UNIQUE (SalaID, RedOznaka, BrojSjedista),  
  
    CONSTRAINT CHK_Sjedista_Broj  
        CHECK (BrojSjedista > 0),  
  
    CONSTRAINT CHK_Sjedista_Tip  
        CHECK (TipSjedista IN (N'Standard', N'VIP', N'Pristupacno'))  
);
```

Slika 9 - Query za kreiranje tabele Sjedista

Ograničenje CHK_Sjedista_Broj osigurava da broj sjedišta bude veći od nule, dok CHK_Sjedista_Tip definiše dozvoljene tipove sjedišta.

3.6. Tabela „Projekcije“

Tabela Projekcije je srce sistema jer povezuje filmove i sale u određenom vremenskom terminu. ProjekcijaID je primarni ključ sa identity svojstvom. FilmID i SalaID su strani ključevi koji povezuju projekciju sa konkretnim filmom i konkretnom salom. Ove kolone su obavezne.

```

CREATE TABLE kino.Projekcije (
    ProjekcijaID INT IDENTITY PRIMARY KEY,
    FilmID INT NOT NULL,
    SalaID INT NOT NULL,
    DatumVrijeme DATETIME NOT NULL,
    Jezik NVARCHAR(30) NOT NULL,
    FormatProjekcije NVARCHAR(15) NOT NULL,
    OsnovnaCijena DECIMAL(10,2) NOT NULL,
    StatusProjekcije NVARCHAR(20) NOT NULL DEFAULT N'Planirana',

    CONSTRAINT FK_Projekcije_Filmovi
        FOREIGN KEY (FilmID)
        REFERENCES kino.Filmovi(FilmID),

    CONSTRAINT FK_Projekcije_Sale
        FOREIGN KEY (SalaID)
        REFERENCES kino.Sale(SalaID),

    CONSTRAINT UQ_Projekcije
        UNIQUE (SalaID, DatumVrijeme),

    CONSTRAINT CHK_Projekcije_Format
        CHECK (FormatProjekcije IN (N'2D', N'3D', N'IMAX')),

    CONSTRAINT CHK_Projekcije_Cijena
        CHECK (OsnovnaCijena > 0),

    CONSTRAINT CHK_Projekcije_Status
        CHECK (
            StatusProjekcije IN
            (N'Planirana', N'U toku', N'Završena', N'Otkazana')
        )
);

```

Slika 10 - Query za kreiranje tabele Projekcije

DatumVrijeme je kolona tipa DATETIME koja definiše tačan termin projekcije. Kombinacija SaleID i DatumVrijeme je jedinstvena, što osigurava da se u istoj sali ne mogu održavati dvije projekcije u isto vrijeme. Jezik definiše na kom jeziku se film prikazuje. FormatProjekcije može imati vrijednosti 2D, 3D ili IMAX. OsnovnaCijena je decimalna kolona koja čuva cijenu karte za ovu projekciju, sa CHECK ograničenjem da mora biti veća od nule. StatusProjekcije prati trenutno stanje projekcije i može imati vrijednosti Planirana, U toku, Završena ili Otkazana.

3.7. Tabela „Kupci“

Tabela Kupci čuva podatke o registrovanim korisnicima bioskopa. KupacID je primarni ključ sa identity svojstvom. Ime i Prezime su obavezna polja. Email adresa je obavezna i jedinstvena, što je važno za autentifikaciju korisnika u online sistemu. CHECK ograničenje na email adresi osigurava da adresa sadrži @ i tačku, što je osnovna provjera ispravnosti. Telefon je opciono polje. DatumRegistracije ima podrazumijevanu vrijednost GETDATE(), što znači da se prilikom unosa novog kupca automatski upisuje trenutni datum i vrijeme. BrojPosjeta je numeričko polje sa podrazumijevanom vrijednošću 0, koje se automatski povećava svaki put kada kupac izvrši novu rezervaciju.

```
CREATE TABLE prodaja.Kupci (  
    KupacID INT IDENTITY PRIMARY KEY,  
    Ime NVARCHAR(50) NOT NULL,  
    Prezime NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100) NOT NULL UNIQUE,  
    Telefon NVARCHAR(25),  
    DatumRegistracije DATETIME NOT NULL DEFAULT GETDATE(),  
    BrojPosjeta INT NOT NULL DEFAULT 0,  
  
    CONSTRAINT CHK_Kupci_Email  
        CHECK (Email LIKE N'%_@_%._%'),  
  
    CONSTRAINT CHK_Kupci_Posjete  
        CHECK (BrojPosjeta >= 0)  
);
```

Slika 11 - Query za kreiranje tabele Kupci

3.8. Tabela „Rezervacije“

Tabela Rezervacije predstavlja centralni entitet za praćenje rezervacija karata. RezervacijaID je primarni ključ sa identity svojstvom. KupacID i ProjekcijaID su strani ključevi koji povezuju rezervaciju sa konkretnim kupcem i konkretnom projekcijom. Ove kolone su obavezne. DatumRezervacije ima podrazumijevanu vrijednost GETDATE(). StatusRezervacije može imati vrijednosti Na cekanju, Potvrđena, Otkazana ili Iskoristena. RokPotvrde definiše do kada rezervacija mora biti potvrđena. Napomena je opciono polje za dodatne informacije.


```

CREATE TABLE prodaja.Rezervacije (
    RezervacijaID INT IDENTITY PRIMARY KEY,
    KupacID INT NOT NULL,
    ProjekcijaID INT NOT NULL,
    DatumRezervacije DATETIME NOT NULL DEFAULT GETDATE(),
    StatusRezervacije NVARCHAR(20) NOT NULL DEFAULT N'Potvrđena',
    RokPotvrde DATETIME,
    Napomena NVARCHAR(300),

    CONSTRAINT FK_Rezervacije_Kupci
        FOREIGN KEY (KupacID)
        REFERENCES prodaja.Kupci(KupacID),

    CONSTRAINT FK_Rezervacije_Projekcije
        FOREIGN KEY (ProjekcijaID)
        REFERENCES kino.Projekcije(ProjekcijaID),

    CONSTRAINT CHK_Rezervacije_Status
        CHECK (
            StatusRezervacije IN
            (N'Na cekanju', N'Potvrđena', N'Otkazana', N'Iskoristena')
        )
);

```

Slika 12 - Query za kreiranje tabele Rezervacije

3.9. Tabela „RezervisanaSjedista“

Tabela RezervisanaSjedista predstavlja vezu više-prema-više između rezervacija i sjedišta. Jedna rezervacija može uključivati više sjedišta, a jedno sjedište može biti rezervisano u više različitih projekcija. RezervisanoSjedisteID je primarni ključ sa identity svojstvom. RezervacijaID i SjedisteID su strani ključevi. Kombinacija RezervacijaID i SjedisteID je jedinstvena, što sprečava dupliranje istog sjedišta u okviru iste rezervacije. Cijena je obavezna i mora biti veća od nule, što omogućava da se cijena pojedinačnog sjedišta evidentira u trenutku rezervacije.

```

CREATE TABLE prodaja.RezervisanaSjedista (
    RezervisanoSjedisteID INT IDENTITY PRIMARY KEY,
    RezervacijaID INT NOT NULL,
    SjedisteID INT NOT NULL,
    Cijena DECIMAL(10,2) NOT NULL,

    CONSTRAINT FK_RS_Rezervacije
        FOREIGN KEY (RezervacijaID)
        REFERENCES prodaja.Rezervacije(RezervacijaID),

    CONSTRAINT FK_RS_Sjedista
        FOREIGN KEY (SjedisteID)
        REFERENCES kino.Sjedista(SjedisteID),

    CONSTRAINT UQ_RS
        UNIQUE (RezervacijaID, SjedisteID),

    CONSTRAINT CHK_RS_Cijena
        CHECK (Cijena > 0)
);

```

Slika 13 - Query za kreiranje tabele RezervisanaSjedista

3.10. Tabela „Placanja“

Tabela Placanja služi za evidentiranje svih uplata koje su izvršene za rezervacije. Jedna rezervacija može imati više uplata, na primjer avans prilikom rezervacije i ostatak po preuzimanju karata. PlacanjeID je primarni ključ sa identity svojstvom. RezervacijaID je strani ključ koji povezuje uplatu sa konkretnom rezervacijom. DatumPlacanja ima podrazumijevanu vrijednost GETDATE(). Iznos je obavezan i mora biti veći od nule. NacinPlacanja može imati vrijednosti Gotovina, Kartica, Online ili Vaucer. StatusPlacanja može imati vrijednosti Na cekanju, Placeno, Odbijeno ili Refundirano. BrojTransakcije je opciono polje koje čuva identifikator transakcije iz eksternog sistema za plaćanje, sa jedinstvenim indeksom koji sprečava dupliranje.

```
CREATE TABLE prodaja.Placanja (  
    PlacanjeID INT IDENTITY PRIMARY KEY,  
    RezervacijaID INT NOT NULL,  
    DatumPlacanja DATETIME NOT NULL DEFAULT GETDATE(),  
    Iznos DECIMAL(10,2) NOT NULL,  
    NacinPlacanja NVARCHAR(20) NOT NULL,  
    StatusPlacanja NVARCHAR(20) NOT NULL DEFAULT N'Placeno',  
    BrojTransakcije NVARCHAR(60),  
  
    CONSTRAINT FK_Placanja_Rezervacije  
        FOREIGN KEY (RezervacijaID)  
        REFERENCES prodaja.Rezervacije(RezervacijaID),  
  
    CONSTRAINT CHK_Placanja_Iznos  
        CHECK (Iznos > 0),  
  
    CONSTRAINT CHK_Placanja_Nacin  
        CHECK (  
            NacinPlacanja IN  
            (N'Gotovina', N'Kartica', N'Online', N'Vaucer')  
        ),  
  
    CONSTRAINT CHK_Placanja_Status  
        CHECK (  
            StatusPlacanja IN  
            (N'Na cekanju', N'Placeno', N'Odbijeno', N'Refundirano')  
        )  
);
```

Slika 14 - Query za kreiranje tabele Placanja

3.11. Tabela „Cjenovnik“

Tabela Cjenovnik definiše cijene karata u zavisnosti od tipa sale i dana u sedmici. CjenovnikID je primarni ključ sa identity svojstvom. Naziv je obavezan. TipSale definiše na koju vrstu sale se cijena odnosi. DanUSedmici je opciono polje koje omogućava definisanje cijena za određene dane u sedmici, na primjer popust utorkom. Cijena je obavezna i mora biti veća od nule. VaziOd i VaziDo definišu period važenja cijene. Ograničenje CHK_Cjenovnik_Datumi osigurava da je VaziDo, ako nije NULL, veći ili jednak VaziOd.

```
CREATE TABLE prodaja.Cjenovnik (  
    CjenovnikID INT IDENTITY PRIMARY KEY,  
    Naziv NVARCHAR(80) NOT NULL,  
    TipSale NVARCHAR(20) NOT NULL,  
    DanUSedmici INT NULL,  
    Cijena DECIMAL(10,2) NOT NULL,  
    VaziOd DATE NOT NULL,  
    VaziDo DATE,  
  
    CONSTRAINT CHK_Cjenovnik_Cijena  
        CHECK (Cijena > 0),  
  
    CONSTRAINT CHK_Cjenovnik_Dan  
        CHECK (  
            DanUSedmici IS NULL  
            OR DanUSedmici BETWEEN 1 AND 7  
        ),  
  
    CONSTRAINT CHK_Cjenovnik_Datumi  
        CHECK (  
            VaziDo IS NULL  
            OR VaziDo >= VaziOd  
        )  
);
```

Slika 15 - Query za kreiranje tabele Cjenovnik

3.12. Tabela „Zaposleni“

Tabela Zaposleni čuva podatke o zaposlenima u bioskopu. ZaposleniID je primarni ključ sa identity svojstvom. Ime, Prezime, Pozicija i DatumZaposlenja su obavezna polja. Plata je obavezna i mora biti veća od nule. Aktivan je logičko polje sa podrazumijevanom vrijednošću 1.

```
CREATE TABLE kino.Zaposleni (  
    ZaposleniID INT IDENTITY PRIMARY KEY,  
    Ime NVARCHAR(50) NOT NULL,  
    Prezime NVARCHAR(50) NOT NULL,  
    Pozicija NVARCHAR(40) NOT NULL,  
    DatumZaposlenja DATE NOT NULL,  
    Plata DECIMAL(10,2) NOT NULL,  
    Aktivan BIT NOT NULL DEFAULT 1,  
  
    CONSTRAINT CHK_Zaposleni_Plata  
        CHECK (Plata > 0)  
);
```

Slika 16 - Query za kreiranje tabele Zaposleni

3.13. Tabela „Smjene“

Tabela Smjene prati raspored rada zaposlenih. SmjenaID je primarni ključ sa identity svojstvom. ZaposleniID je strani ključ koji povezuje smjenu sa konkretnim zaposlenim. DatumSmjene definiše na koji dan se smjena odnosi. Pocetak i Kraj su DATETIME kolone koje definišu početak i kraj smjene. Ograničenje CHK_Smjene_Vrijeme osigurava da je vrijeme kraja veće od vremena početka.

```
CREATE TABLE kino.Smjene (  
    SmjenaID INT IDENTITY PRIMARY KEY,  
    ZaposleniID INT NOT NULL,  
    DatumSmjene DATE NOT NULL,  
    Pocetak DATETIME NOT NULL,  
    Kraj DATETIME NOT NULL,  
  
    CONSTRAINT FK_Smjene_Zaposleni  
        FOREIGN KEY (ZaposleniID)  
        REFERENCES kino.Zaposleni(ZaposleniID),  
  
    CONSTRAINT CHK_Smjene_Vrijeme  
        CHECK (Kraj > Pocetak)  
);
```

Slika 17 - Query za kreiranje tabele Smjene

4. KONTROLA I INTEGRITET PODATAKA

Integritet podataka predstavlja jedan od najvažnijih koncepata u projektovanju baza podataka. On osigurava da su podaci u bazi tačni, konzistentni i da zadovoljavaju sve poslovne zahtjeve. U ovoj bazi podataka uspostavljena su tri nivoa integriteta: integritet entiteta, integritet domena i referencijalni integritet.

4.1. Integritet na nivou entiteta

Integritet entiteta osigurava da svaki zapis u tabeli bude jedinstven i jasno identifikovan, čime se sprečava pojava duplih podataka koji bi mogli predstavljati isti entitet. U bazi podataka Bioskop, integritet entiteta postiže se definisanjem primarnih ključeva za sve glavne tabele. Primarni ključevi su postavljeni na kolone ZanrID, KategorijaID, FilmID, SalaID, SjedisteID, ProjekcijaID, KupacID, RezervacijaID, RezervisanoSjedisteID, PlacanjeID, CjenovnikID, ZaposleniID i SmjenaID. Ove kolone koriste IDENTITY svojstvo, što omogućava automatsko generisanje jedinstvenih vrijednosti pri unosu novih zapisa.

4.2. Integritet na nivou domena

Integritet domena se odnosi na ograničenja koja definišu dozvoljene vrijednosti za pojedine kolone. Ovaj oblik integriteta osigurava da podaci unutar baze budu validni i konzistentni. U praksi se postiže korišćenjem CHECK ograničenja, koja postavljaju konkretna pravila za unos podataka. U tabeli Filmovi, CHECK ograničenja osiguravaju da trajanje filma bude između 20 i 400 minuta, da godina bude između 1900 i 2100, te da prosječna ocjena bude između 1 i 10. U tabeli Sale, CHECK ograničenja definišu dozvoljene tipove sala i osiguravaju da kapacitet bude pozitivan. U tabeli Projekcije, CHECK ograničenja definišu dozvoljene formate projekcije, osiguravaju da cijena bude veća od nule i definišu dozvoljene statusе projekcije.

4.3. Ograničenja za tabelu Kupci

Za tabelu Kupci, dodato je CHECK ograničenje koje provjerava format email adrese. Ograničenje koristi LIKE operator sa šablonom koji osigurava da email sadrži simbol @ i tačku nakon njega,

čime se sprječava unos očigledno neispravnih email adresa. Također, CHECK ograničenje na koloni BrojPosjeta osigurava da broj posjeta ne bude negativan.

```
CONSTRAINT CHK_Kupci_Email CHECK(Email LIKE N'%@%._%'),  
CONSTRAINT CHK_Kupci_Posjete CHECK(BrojPosjeta>=0)
```

4.4. Ograničenja za tabelu Placanja

Za tabelu Placanja, uspostavljena su tri CHECK ograničenja. Prvo ograničenje provjerava da je iznos uplate veći od nule. Drugo ograničenje definiše dozvoljene načine plaćanja. Treće ograničenje definiše dozvoljene statuse plaćanja.

```
CONSTRAINT CHK_Placanja_Iznos CHECK(Iznos>0),  
CONSTRAINT CHK_Placanja_Nacin CHECK(NacinPlacanja  
IN(N'Gotovina',N'Kartica',N'Online',N'Vaucer')),  
CONSTRAINT CHK_Placanja_Status CHECK(StatusPlacanja IN(N'Na  
cekanju',N'Placeno',N'Odbijeno',N'Refundirano'))
```

4.5. Ograničenja za tabelu Rezervacije

Za tabelu Rezervacije, uspostavljeno je CHECK ograničenje koje definiše dozvoljene statuse rezervacije. Ovo ograničenje osigurava da status rezervacije može biti samo jedna od predviđenih vrijednosti, što je važno za pravilno praćenje životnog ciklusa svake rezervacije.

```
CONSTRAINT CHK_Rezervacije_Status CHECK(StatusRezervacije IN(N'Na  
cekanju',N'Potvrđjena',N'Otkazana',N'Iskoristena'))
```

4.6. Ograničenja za tabelu RezervisanaSjedista

Tabela RezervisanaSjedista sadrži CHECK ograničenje koje osigurava da je cijena pojedinačnog sjedišta veća od nule. Ovo je važno jer se cijena evidentira u trenutku rezervacije i ne smije biti negativna ili nula.

CONSTRAINT CHK_RS_Cijena CHECK(Cijena>0)

4.7. Ograničenja za tabelu Cjenovnik

Tabela Cjenovnik ima dva CHECK ograničenja. Prvo osigurava da cijena bude veća od nule. Drugo provjerava da je dan u sedmici, ako je naveden, između 1 i 7, što odgovara ponedjeljku do nedjelje. Također, ograničenje osigurava da je datum do kojeg cijena važi, ako je naveden, veći ili jednak datumu od kojeg cijena važi.

**CONSTRAINT CHK_Cjenovnik_Cijena CHECK(Cijena>0),
CONSTRAINT CHK_Cjenovnik_Dan CHECK(DanUSedmici IS NULL OR DanUSedmici
BETWEEN 1 AND 7),
CONSTRAINT CHK_Cjenovnik_Datumi CHECK(VaziDo IS NULL OR
VaziDo>=VaziOd)**

4.8. Ograničenja za tabelu Smjene

Tabela Smjene ima CHECK ograničenje koje osigurava da je vrijeme kraja smjene veće od vremena početka smjene, što je logički neophodno za ispravan unos smjena.

CONSTRAINT CHK_Smjene_Vrijeme CHECK(Kraj>Pocetak)

Primjenom integriteta na nivou domena smanjuje se rizik od pogrešnog unosa podataka i obezbjeđuje se visok nivo kvaliteta i pouzdanosti informacija unutar sistema.

4.9. Referencijalni integritet u bazi

Referencijalni integritet osigurava da veze između tabela budu konzistentne. U bazi Bioskop, referencijalni integritet je uspostavljen kroz više stranih ključeva. Tabela Filmovi sadrži dva strana ključa, ZavrID koji referencira tabelu Zavrrovi i KategorijaID koji referencira tabelu Kategorije. Ovo znači da se film može kreirati samo sa postojećim žanrom i postojećom kategorijom.

Tabela Sjedista sadrži strani ključ SalaID koji referencira tabelu Sale. Ovo osigurava da se sjedište može kreirati samo za postojeću salu. Tabela Projekcije sadrži dva strana ključa. Prvi je FilmID koji referencira tabelu Filmovi, što znači da se projekcija može kreirati samo za postojeći film. Drugi je SalaID koji referencira tabelu Sale, što znači da se projekcija može kreirati samo za postojeću salu.

Tabela Rezervacije sadrži dva strana ključa. Prvi je KupacID koji referencira tabelu Kupci, što znači da se rezervacija može kreirati samo za postojećeg kupca. Drugi je ProjekcijaID koji referencira tabelu Projekcije, što znači da se rezervacija može kreirati samo za postojeću projekciju.

Tabela RezervisanaSjedista sadrži dva strana ključa. Prvi je RezervacijaID koji referencira tabelu Rezervacije, što znači da se sjedište može rezervisati samo za postojeću rezervaciju. Drugi je SjedisteID koji referencira tabelu Sjedista, što znači da se može rezervisati samo postojeće sjedište.

Tabela Placanja sadrži strani ključ RezervacijaID koji referencira tabelu Rezervacije, što osigurava da se plaćanje može evidentirati samo za postojeću rezervaciju. Tabela Smjene sadrži strani ključ ZaposleniID koji referencira tabelu Zaposleni, što osigurava da se smjena može kreirati samo za postojećeg zaposlenog.

5. KREIRANJE INDEKSA ZA OPTIMIZACIJU PERFORMANSI

Indeksi su posebne strukture u bazi podataka koje značajno ubrzavaju izvršavanje upita, posebno onih koji pretražuju, filtriraju ili sortiraju velike količine podataka. Razlog zašto indeksi ubrzavaju pretragu sličan je razlogu zašto indeks na kraju knjige pomaže da brzo pronađemo željeno poglavlje umjesto da listamo cijelu knjigu.

5.1. Indeks na koloni Naziv u tabeli Filmovi

Prvi indeks je kreiran na koloni Naziv u tabeli Filmovi. Ovaj indeks je izuzetno važan jer će se filmovi često pretraživati po nazivu prilikom prikaza rasporeda ili prilikom pretrage filmova od strane korisnika. Bez indeksa, SQL Server bi morao pregledati svaki red u tabeli da bi pronašao film sa traženim nazivom, što bi bilo sporo kada tabela naraste na stotine ili hiljade filmova.

```
CREATE INDEX IX_Filmovi_Naziv ON kino.Filmovi(Naziv)
```

5.2. Indeks na koloni DatumVrijeme u tabeli Projekcije

Drugi indeks je kreiran na koloni DatumVrijeme u tabeli Projekcije. Ovaj indeks je važan jer će sistem često izvršavati upite koji pronalaze projekcije za određeni period, kao što je prikaz svih projekcija za tekući dan ili provjera rasporeda za narednih nekoliko dana. Indeks na datumskoj koloni omogućava da se ovi upiti izvršavaju trenutno, čak i kada tabela sadrži veliki broj zapisa.

```
CREATE INDEX IX_Projekcije_Datum ON kino.Projekcije(DatumVrijeme)
```

5.3. Kompozitni indeks na kolonama FilmID i StatusProjekcije u tabeli Projekcije

Treći indeks je kompozitni indeks na kolonama FilmID i StatusProjekcije u tabeli Projekcije. Ovaj indeks je namijenjen upitima koji pretražuju projekcije određenog filma sa određenim statusom. Kompozitni indeksi su posebno efikasni kada se u WHERE dijelu upita koriste dvije kolone.

```
CREATE INDEX IX_Projekcije_FilmStatus ON kino.Projekcije(FilmID,StatusProjekcije)
```

5.4. Indeks na kolonama SalaID i RedOznaka u tabeli Sjedista

Četvrti indeks je kreiran na kolonama SalaID i RedOznaka u tabeli Sjedista. Ovaj indeks je važan za upite koji pretražuju sjedišta po sali i redu, što je česta operacija prilikom prikaza rasporeda sjedišta za određenu projekciju.

CREATE INDEX IX_Sjedista_Sala ON kino.Sjedista(SalaID,RedOznaka)

5.5. Indeks na koloni KupacID u tabeli Rezervacije

Peti indeks je kreiran na koloni KupacID u tabeli Rezervacije. Ovaj indeks je važan za upite koji pretražuju rezervacije određenog kupca, kao što je prikaz istorije rezervacija ili provjera aktivnih rezervacija.

CREATE INDEX IX_Rezervacije_Kupac ON prodaja.Rezervacije(KupacID)

5.6. Kompozitni indeks na kolonama ProjekcijaID i StatusRezervacije u tabeli Rezervacije

Šesti indeks je kompozitni indeks na kolonama ProjekcijaID i StatusRezervacije u tabeli Rezervacije. Ovaj indeks je namijenjen upitima koji pretražuju rezervacije za određenu projekciju sa određenim statusom, što je korisno za operativni rad.

CREATE INDEX IX_Rezervacije_ProjekcijaStatus ON prodaja.Rezervacije(ProjekcijaID,StatusRezervacije)

5.7. Indeks na koloni DatumPlacanja u tabeli Placanja

Sedmi indeks je kreiran na koloni DatumPlacanja u tabeli Placanja. Ovaj indeks je važan za finansijske izvještaje koji analiziraju prihode po vremenskim periodima.

CREATE INDEX IX_Placanja_Datum ON prodaja.Placanja(DatumPlacanja)

5.8. Filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja

Osmi indeks je filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja. Ovaj indeks je poseban jer se primjenjuje samo na redove gdje BrojTransakcije nije NULL. Ovakav indeks sprečava dupliranje brojeva transakcija, što je važno za praćenje online plaćanja, dok istovremeno dozvoljava više NULL vrijednosti.

CREATE UNIQUE INDEX UX_Placanja_Transakcija ON prodaja.Placanja(BrojTransakcije) WHERE BrojTransakcije IS NOT NULL

6. UNOS PODATAKA U BAZU PODATAKA

Nakon što je struktura baze podataka u potpunosti definisana, potrebno je unijeti test podatke koji će omogućiti demonstraciju funkcionalnosti sistema. Test podaci su odabrani tako da pokriju različite scenarije koji se mogu pojaviti u stvarnom radu sistema.

6.1. Unos podataka u tabelu Zanrovi

U tabelu Zanrovi unosi se osam različitih žanrova koji pokrivaju uobičajene kategorije filmova. Unose se Akcija, Drama, Komediya, Animirani, Triler, Naucna fantastika, Dokumentarni i Avantura. Svaki žanr ima kratak opis koji pojašnjava njegovu prirodu.

```
INSERT INTO sif.Zanrovi (Naziv, Opis)
VALUES
    (N'Akcija', N'Dinamicni filmovi'),
    (N'Drama', N'Dramski filmovi'),
    (N'Komediya', N'Humoristicki filmovi'),
    (N'Animirani', N'Animirani filmovi'),
    (N'Triler', N'Napeti filmovi'),
    (N'Naucna fantastika', N'Futuristicki filmovi'),
    (N'Dokumentarni', N'Dokumentarni sadrzaj'),
    (N'Avantura', N'Avanturisticki filmovi');
```

Slika 18 - Unos podataka u tabelu Zanrovi

6.2. Unos podataka u tabelu Kategorije

U tabelu Kategorije unosi se pet dobnih kategorija koje definišu starosne granice za gledanje filmova. Unose se Svi uzrasti sa minimalnom dobi 0, 7+ sa minimalnom dobi 7, 12+ sa minimalnom dobi 12, 15+ sa minimalnom dobi 15 i 18+ sa minimalnom dobi 18. Ove kategorije su standardne u filmskoj industriji.

```
INSERT INTO sif.Kategorije (Naziv, MinimalnaDob)
VALUES
    (N'Svi uzrasti', 0),
    (N'7+', 7),
    (N'12+', 12),
    (N'15+', 15),
    (N'18+', 18);
```

Slika 19 - Unos podataka u tabelu Kategorije

6.3. Unos podataka u tabelu Sale

U tabelu Sale unose se četiri sale različitih tipova i kapaciteta. Sala 1 je standardna sala sa kapacitetom od 40 sjedišta. Sala 2 je 3D sala sa kapacitetom od 30 sjedišta. Sala 3 je IMAX sala sa kapacitetom od 50 sjedišta. VIP salon je VIP sala sa kapacitetom od 20 sjedišta. Svaka sala ima datum otvaranja i označena je kao aktivna.

```
INSERT INTO kino.Sale (Naziv, TipSale, Kapacitet, DatumOtvaranja)
VALUES
    (N'Sala 1', N'Standard', 40, '2020-01-10'),
    (N'Sala 2', N'3D', 30, '2020-01-10'),
    (N'Sala 3', N'IMAX', 50, '2021-05-01'),
    (N'VIP salon', N'VIP', 20, '2022-09-15');
```

Slika 20 - Unos podataka u tabelu Sale

6.4. Unos podataka u tabelu Filmovi

U tabelu Filmovi unosi se deset filmova sa različitim karakteristikama. Filmovi su raspoređeni po različitim žanrovima i kategorijama, sa različitim trajanjima, godinama i prosječnim ocjenama. Unose se filmovi kao što su Posljednja misija, Tihi grad, Vikend bez plana, Zvezdani prijatelji, Sjena u prolazu, Nova orbita, Planeta voda, Izgubljena mapa, Brzina odluke i Komšije na odmoru. Svaki film ima svoj originalni naziv, datum premijere i oznaku da je aktivan.

```
INSERT INTO kino.Filmovi (
    ZanzID,
    KategorijaID,
    Naziv,
    OriginalniNaziv,
    TrajanjeMin,
    Godina,
    DatumPremijere,
    ProsjecnaOcjena
)
VALUES
    (
        1, 3, N'Posljednja misija', N'The Last Mission',
        125, 2026, '2026-07-10', 8.2
    ),
    (
        2, 3, N'Tihi grad', N'Silent City',
        112, 2026, '2026-06-15', 7.9
    ),
    (
        3, 2, N'Vikend bez plana', N'Weekend Unplanned',
        98, 2026, '2026-08-01', 7.4
    ),
    (
        4, 1, N'Zvezdani prijatelji', N'Star Friends',
        91, 2026, '2026-05-20', 8.5
    ).
```

Slika 21 - Unos podataka u tabelu Filmovi

6.5. Unos podataka u tabelu Sjedišta

U tabelu Sjedišta unose se sjedišta za svaku salu. Za svaku salu, sjedišta su organizovana po redovima, pri čemu svaki red ima određeni broj sjedišta. U sali 1, redovi A, B, C i D imaju po deset sjedišta, sa prva dva sjedišta u redu A označena kao pristupačna. U sali 2, redovi A, B i C imaju po deset sjedišta. U sali 3, redovi A, B, C, D i E imaju po deset sjedišta. U VIP salonu, redovi A i B imaju po deset VIP sjedišta. Ukupno je uneseno 140 sjedišta.

```
INSERT INTO kino.Sjedišta (  
    SalaID,  
    RedOznaka,  
    BrojSjedišta,  
    TipSjedišta  
)  
VALUES  
    (1, N'A', 1, N'Pristupacno'),  
    (1, N'A', 2, N'Pristupacno'),  
    (1, N'A', 3, N'Standard'),  
    (1, N'A', 4, N'Standard'),  
    (1, N'A', 5, N'Standard'),  
    (1, N'A', 6, N'Standard'),  
    (1, N'A', 7, N'Standard'),  
    (1, N'A', 8, N'Standard'),  
    (1, N'A', 9, N'Standard'),  
    (1, N'A', 10, N'Standard'),  
  
    (1, N'B', 1, N'Standard'),  
    (1, N'B', 2, N'Standard'),
```

Slika 22 - Unos podataka u tabelu Sjedišta

6.6. Unos podataka u tabelu Kupci

U tabelu Kupci unosi se dvadeset kupaca sa različitim podacima. Kupci imaju različite brojeve posjeta, od nula do jedanaest, što će omogućiti testiranje funkcije za izračunavanje popusta na osnovu broja posjeta. Svaki kupac ima svoje ime, prezime, email adresu i telefonski broj.

```
INSERT INTO prodaja.Kupci (  
    Ime,  
    Prezime,  
    Email,  
    Telefon,  
    BrojPosjeta  
)  
VALUES  
    ('Marko', 'Markovic', 'marko01@example.com', '065111111', 5),  
    ('Ana', 'Jovanovic', 'ana02@example.com', '066222222', 3),  
    ('Nikola', 'Petrovic', 'nikola03@example.com', '067333333', 8),  
    ('Jelena', 'Ilic', 'jelena04@example.com', '065444444', 1),  
    ('Stefan', 'Savic', 'stefan05@example.com', '066555555', 4),  
    ('Milica', 'Kovacevic', 'milica06@example.com', '067666666', 6),  
    ('Dejan', 'Popovic', 'dejan07@example.com', '065777777', 9),
```

Slika 23 - Unos podataka u tabelu Kupci

6.7. Unos podataka u tabelu Zaposleni

U tabelu Zaposleni unosi se šest zaposlenih sa različitim pozicijama. Unose se blagajnici, menadžer, operateri projekcija i kontrolor. Svaki zaposleni ima svoje ime, prezime, poziciju, datum zaposlenja i platu. Svi zaposleni su označeni kao aktivni.

```
INSERT INTO kino.Zaposleni (  
    Ime,  
    Prezime,  
    Pozicija,  
    DatumZaposlenja,  
    Plata  
)  
VALUES  
    ('Milan', 'Jovic', 'Blagajnik', '2022-01-10', 1400),  
    ('Sara', 'Kostic', 'Menadzer', '2020-03-01', 2200),  
    ('Petar', 'Vasic', 'Operater projekcije', '2021-06-15', 1700),
```

Slika 24 - Unos podataka u tabelu Zaposleni

6.8. Unos podataka u tabelu Smjene

U tabelu Smjene unose se smjene za zaposlene za početak septembra 2026. godine. Svaka smjena ima datum, vrijeme početka i vrijeme kraja. Smjene su raspoređene tako da pokrivaju različite radne periode tokom dana.

```
INSERT INTO kino.Smjene (  
    ZaposleniID,  
    DatumSmjene,  
    Pocetak,  
    Kraj  
)  
VALUES  
(1, '2026-09-01', '2026-09-01T14:00', '2026-09-01T22:00'),  
(2, '2026-09-01', '2026-09-01T12:00', '2026-09-01T20:00'),  
(3, '2026-09-01', '2026-09-01T16:00', '2026-09-02T00:00'),  
(4, '2026-09-02', '2026-09-02T14:00', '2026-09-02T22:00'),  
(5, '2026-09-02', '2026-09-02T16:00', '2026-09-03T00:00'),  
(6, '2026-09-02', '2026-09-02T15:00', '2026-09-02T23:00');
```

Slika 25 - Unos podataka u tabelu Smjene

6.9. Unos podataka u tabelu Cjenovnik

U tabelu Cjenovnik unose se cijene karata za različite tipove sala. Standardna karta košta 8, 3D karta košta 11, IMAX karta košta 15, a VIP karta košta 20. Također, unosi se posebna cijena za utorak popust koja iznosi 6 za standardnu salu. Sve cijene važe od 1. januara 2026. godine.

```
INSERT INTO prodaja.Cjenovnik (  
    Naziv,  
    TipSale,  
    DanUSedmici,  
    Cijena,  
    VaziOd,  
    VaziDo  
)  
VALUES  
(N'Standard radni dan', N'Standard', NULL, 8, '2026-01-01', NULL),  
(N'3D karta', N'3D', NULL, 11, '2026-01-01', NULL),  
(N'IMAX karta', N'IMAX', NULL, 15, '2026-01-01', NULL),  
(N'VIP karta', N'VIP', NULL, 20, '2026-01-01', NULL),  
(N'Utorak popust', N'Standard', 3, 6, '2026-01-01', NULL);
```

Slika 26 - Unos podataka u tabelu Cjenovnik

6.10. Unos podataka u tabelu Projekcije

U tabelu Projekcije unosi se trideset projekcija za period od 1. do 8. septembra 2026. godine. Svaki dan ima po četiri projekcije u različitim salama, sa početkom u 16, 18, 20 i 22 sata. Projekcije su raspoređene tako da se filmovi smjenjuju po salama. Prve četiri projekcije su označene kao završene, dok su ostale planirane.

```
INSERT INTO kino.Projekcije (  
    FilmID,  
    SalaID,  
    DatumVrijeme,  
    Jezik,  
    FormatProjekcije,  
    OsnovnaCijena,  
    StatusProjekcije  
)  
VALUES  
(1, 1, '2026-09-01T16:00:00', N'BHS', N'2D', 8.00, N'Završena'),  
(2, 2, '2026-09-01T18:00:00', N'BHS', N'3D', 11.00, N'Završena'),  
(3, 3, '2026-09-01T20:00:00', N'BHS', N'IMAX', 15.00, N'Završena'),  
(4, 4, '2026-09-01T22:00:00', N'BHS', N'2D', 20.00, N'Završena'),  
(5, 1, '2026-09-02T16:00:00', N'BHS', N'2D', 8.00, N'Planirana'),  
(6, 2, '2026-09-02T18:00:00', N'BHS', N'3D', 11.00, N'Planirana'),
```

Slika 27 - Unos podataka u tabelu Projekcije

6.11. Unos podataka u tabelu Rezervacije

U tabelu Rezervacije unosi se trideset pet rezervacija za različite projekcije. Rezervacije su kreirane od strane različitih kupaca, sa različitim datumima rezervacije. Sve rezervacije su označene kao potvrđene, sa rokom potvrde postavljenim na dan projekcije u podne. Ove rezervacije pokrivaju različite projekcije i omogućavaju testiranje različitih upita.

```
INSERT INTO prodaja.Rezervacije (  
    KupacID,  
    ProjekcijaID,  
    DatumRezervacije,  
    StatusRezervacije,  
    RokPotvrde  
)  
VALUES  
(1, 1, '2026-08-02', N'Potvrđjena', '2026-09-02T12:00:00'),  
(2, 2, '2026-08-03', N'Potvrđjena', '2026-09-03T12:00:00'),  
(3, 3, '2026-08-04', N'Potvrđjena', '2026-09-04T12:00:00'),  
(4, 4, '2026-08-05', N'Potvrđjena', '2026-09-05T12:00:00'),  
(5, 5, '2026-08-06', N'Potvrđjena', '2026-09-06T12:00:00'),
```

Slika 28 - Unos podataka u tabelu Rezervacije

6.12. Unos podataka u tabelu RezervisanaSjedista

U tabelu RezervisanaSjedista unose se sjedišta za svaku rezervaciju. Svaka rezervacija ima po jedno sjedište, sa cijenom koja odgovara cijeni karte za tu projekciju. Sjedišta su raspoređena po različitim redovima i salama, što omogućava testiranje upita za slobodna sjedišta.

```
INSERT INTO prodaja.RezervisanaSjedista (  
    RezervacijaID,  
    SjedisteID,  
    Cijena  
)  
VALUES  
    (1, 3, 8.00),  
    (2, 6, 11.00),  
    (3, 9, 15.00),  
    (4, 12, 20.00),
```

Slika 29 - Unos podataka u tabelu RezervisanaSjedista

6.13. Unos podataka u tabelu Placanja

U tabelu Placanja unose se uplate za svaku rezervaciju. Uplate su izvršene različitim načinima plaćanja, uključujući gotovinu, karticu, online plaćanje i vaucher. Sve uplate su označene kao plaćene, sa odgovarajućim datumima plaćanja. Za online i kartične transakcije, evidentiran je i broj transakcije.

```
INSERT INTO prodaja.Placanja (  
    RezervacijaID,  
    DatumPlacanja,  
    Iznos,  
    NacinPlacanja,  
    StatusPlacanja,  
    BrojTransakcije  
)  
VALUES  
    (1, '2026-08-02', 8.00, N'Gotovina', N'Placeno', NULL),  
    (2, '2026-08-03', 11.00, N'Kartica', N'Placeno', N'TXN-00002'),  
    (3, '2026-08-04', 15.00, N'Online', N'Placeno', N'TXN-00003'),  
    (4, '2026-08-05', 20.00, N'Vaucer', N'Placeno', N'TXN-00004'),  
    (5, '2026-08-06', 8.00, N'Gotovina', N'Placeno', NULL),
```

Slika 30 - Unos podataka u tabelu Placanja

7. KREIRANJE POGLEDA ZA JEDNOSTAVNIJI PRISTUP PODACIMA

Pogledi su virtuelne tabele koje ne čuvaju podatke fizički, već predstavljaju rezultat unaprijed definisanog upita. Oni pojednostavljuju pristup podacima tako što omogućavaju korisnicima da rade sa jednostavnom strukturom, skrivajući kompleksnost osnovnih tabela i relacija. Također, pogledi povećavaju sigurnost jer mogu prikazati samo određene kolone ili redove, skrivajući osjetljive podatke.

7.1. Pogled vw_RasporedProjekcija

Prvi pogled, nazvan vw_RasporedProjekcija, kreiran je da pruži sveobuhvatan pregled rasporeda projekcija sa svim relevantnim detaljima. Ovaj pogled spaja podatke iz tabela Projekcije, Filmovi, Zanrovi, Kategorije i Sale, prikazujući informacije koje su korisne za operativni rad i prikaz rasporeda.

```
CREATE VIEW izvjestaji.vw_RasporedProjekcija AS
SELECT
    p.ProjekcijaID,
    f.Naziv AS Film,
    z.Naziv AS Zavr,
    k.Naziv AS Kategorija,
    s.Naziv AS Sala,
    s.TipSale,
    p.DatumVrijeme,
    p.Jezik,
    p.FormatProjekcije,
    p.OsnovnaCijena,
    p.StatusProjekcije
FROM kino.Projekcije p
INNER JOIN kino.Filmovi f
    ON p.FilmID = f.FilmID
INNER JOIN sif.Zanrovi z
    ON f.ZanrID = z.ZanrID
INNER JOIN sif.Kategorije k
    ON f.KategorijaID = k.KategorijaID
INNER JOIN kino.Sale s
    ON p.SalaID = s.SalaID;
GO
```

Slika 31 - Query za kreiranje pogleda vw_RasporedProjekcija

Pogled prikazuje identifikator projekcije, naziv filma, žanr, kategoriju, naziv sale, tip sale, datum i vrijeme projekcije, jezik, format projekcije, osnovnu cijenu i status projekcije. Kreiranjem ovog pogleda, korisnici ne moraju svaki put pisati kompleksne JOIN upite da bi dobili potrebne

informacije. Umjesto toga, mogu jednostavno izvršiti SELECT upit nad ovim pogledom i dobiti sve relevantne podatke o rasporedu.

7.2. Pogled vw_RezervacijeDetalji

Drugi pogled, nazvan vw_RezervacijeDetalji, kreiran je za potrebe operativnog praćenja rezervacija. Ovaj pogled prikazuje detaljne informacije o svakoj rezervaciji, uključujući podatke o kupcu, filmu, sali, broju karata i ukupnoj cijeni.

```
CREATE VIEW izvjestaji.vw_RezervacijeDetalji AS
SELECT
    r.RezervacijaID,
    k.Ime + N' ' + k.Prezime AS Kupac,
    f.Naziv AS Film,
    p.DatumVrijeme,
    s.Naziv AS Sala,
    COUNT(rs.RezervisanoSjedisteID) AS BrojKarata,
    SUM(rs.Cijena) AS UkupnaCijena,
    r.StatusRezervacije
FROM prodaja.Rezervacije r
INNER JOIN prodaja.Kupci k
    ON r.KupacID = k.KupacID
INNER JOIN kino.Projekcije p
    ON r.ProjekcijaID = p.ProjekcijaID
INNER JOIN kino.Filmovi f
    ON p.FilmID = f.FilmID
INNER JOIN kino.Sale s
    ON p.SalaID = s.SalaID
LEFT JOIN prodaja.RezervisanaSjedista rs
    ON r.RezervacijaID = rs.RezervacijaID
GROUP BY
    r.RezervacijaID,
    k.Ime,
    k.Prezime,
    f.Naziv,
    p.DatumVrijeme,
    s.Naziv,
    r.StatusRezervacije;
GO
```

Slika 32 - Query za kreiranje pogleda vw_RezervacijeDetalji

Pogled prikazuje identifikator rezervacije, ime i prezime kupca, naziv filma, datum i vrijeme projekcije, naziv sale, broj karata, ukupnu cijenu i status rezervacije. Korištenje LEFT JOIN-a za tabelu RezervisanaSjedista osigurava da se prikažu i rezervacije koje još nemaju dodijeljena sjedišta, iako u testnim podacima sve rezervacije imaju sjedišta. Grupisanje po rezervaciji omogućava prikaz zbirnih podataka po rezervaciji.

7.3. Pogled vw_PrihodPoFilmu

Treći pogled, nazvan vw_PrihodPoFilmu, kreiran je za potrebe finansijskog izvještavanja. Ovaj pogled prikazuje prihode ostvarene po svakom filmu u bioskopu. Spaja podatke iz tabela Filmovi, Projekcije, Rezervacije i Placanja, grupirajući ih po filmovima.

```
CREATE VIEW izvjestaji.vw_PrihodPoFilmu AS
SELECT
    f.FilmID,
    f.Naziv,
    COUNT(DISTINCT p.ProjekcijaID) AS BrojProjekcija,
    COUNT(DISTINCT pl.PlacanjeID) AS BrojUplata,
    ISNULL(SUM(pl.Iznos), 0) AS Prihod
FROM kino.Filmovi f
LEFT JOIN kino.Projekcije p
    ON f.FilmID = p.FilmID
LEFT JOIN prodaja.Rezervacije r
    ON p.ProjekcijaID = r.ProjekcijaID
LEFT JOIN prodaja.Placanja pl
    ON r.RezervacijaID = pl.RezervacijaID
    AND pl.StatusPlacanja = N'Placeno'
GROUP BY
    f.FilmID,
    f.Naziv;
GO
```

Slika 33 - Query za kreiranje pogleda vw_PrihodPoFilmu

Pogled prikazuje identifikator filma, naziv filma, broj projekcija, broj uplata i ukupan prihod. Korištenje funkcije ISNULL osigurava da filmovi koji nikada nisu imali projekcije ili uplate budu prikazani sa nulnim prihodom, a ne sa NULL vrijednošću. Filtriranje na StatusPlacanja jednako Placeno osigurava da se u obzir uzimaju samo uspješne uplate. Ovaj pogled je posebno koristan za menadžment jer omogućava brz uvid u to koji filmovi donose najviše prihoda.

8. KREIRANJE KORISNIČKIH FUNKCIJA ZA PONOVO KORIŠTENJE KODA

Korisnički definisane funkcije su objekti koji primaju parametre, izvršavaju određenu logiku i vraćaju vrijednost. Za razliku od procedura, funkcije se mogu koristiti unutar SELECT upita, što ih čini vrlo fleksibilnim za ponovno korištenje koda.

8.1. Funkcija fn_PopustKupca

Prva funkcija, nazvana fn_PopustKupca, izračunava popust koji se odobrava kupcu na osnovu broja njegovih prethodnih posjeta. Ova funkcija prima broj posjeta kao parametar, a vraća procenat popusta kao decimalni broj. Logika funkcije je jednostavna. Ako je kupac imao deset ili više posjeta, odobrava mu se popust od dvadeset posto. Ako je imao pet ili više posjeta, popust je deset posto. Ako je imao tri ili više posjeta, popust je pet posto. U suprotnom, popust nije odobren. Ova funkcija je implementirana kao skalarna funkcija, što znači da vraća jednu vrijednost. Može se koristiti unutar SELECT upita za izračunavanje popusta za svakog kupca pojedinačno.

```
CREATE FUNCTION prodaja.fn_PopustKupca
(
    @BrojPosjeta INT
)
RETURNS DECIMAL(5,2)
AS
BEGIN
    RETURN
    CASE
        WHEN @BrojPosjeta >= 10 THEN 20
        WHEN @BrojPosjeta >= 5 THEN 10
        WHEN @BrojPosjeta >= 3 THEN 5
        ELSE 0
    END;
END;
GO
```

Slika 34 - UDF za izračunavanje popusta

8.2. Funkcija fn_SlobodnaSjedista

Druga funkcija, nazvana fn_SlobodnaSjedista, prikazuje sva sjedišta koja su slobodna za određenu projekciju. Ova funkcija prima identifikator projekcije kao parametar, a vraća tabelu sa svim kolonama iz tabele Sjedista koje zadovoljavaju uslove slobodnosti. Ova funkcija je implementirana kao tabelarna funkcija, što znači da vraća skup redova.

```
CREATE FUNCTION kino.fn_SlobodnaSjedista
(
    @ProjekcijaID INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        sj.SjedisteID,
        sj.RedOznaka,
        sj.BrojSjedista,
        sj.TipSjedista
    FROM kino.Projekcije p
    INNER JOIN kino.Sjedista sj
        ON p.SalaID = sj.SalaID
    WHERE
        p.ProjekcijaID = @ProjekcijaID
        AND sj.Aktivno = 1
        AND NOT EXISTS
        (
            SELECT 1
            FROM prodaja.RezervisanaSjedista rs
            INNER JOIN prodaja.Rezervacije r
                ON rs.RezervacijaID = r.RezervacijaID
            WHERE
                r.ProjekcijaID = @ProjekcijaID
                AND rs.SjedisteID = sj.SjedisteID
                AND r.StatusRezervacije IN
                (
                    N'Na cekanju',
                    N'Potvrđjena'
                )
        )
    )
);
GO
```

Slika 35 - UDF za prikaz slobodnih sjedišta

Implementacija koristi složenu logiku za određivanje slobodnosti sjedišta. Sjedište je slobodno ako je aktivno i ako ne postoji nijedna rezervacija za tu projekciju koja uključuje to sjedište, a čiji je status na čekanju ili potvrđena. Ova funkcija je izuzetno korisna za online sistem rezervacija jer omogućava korisnicima da vide koja sjedišta su slobodna za željenu projekciju.

8.3. Funkcija fn_PrihodProjekcije

Treća funkcija, nazvana fn_PrihodProjekcije, izračunava ukupan prihod ostvaren od jedne projekcije. Ova funkcija prima identifikator projekcije kao parametar, a vraća ukupan iznos plaćenih karata za tu projekciju. Funkcija sumira sve uplate koje su povezane sa rezervacijama za datu projekciju, pri čemu se uzimaju u obzir samo uplate sa statusom Placeno.

```
CREATE FUNCTION prodaja.fn_PrihodProjekcije
(
    @ProjekcijaID INT
)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @x DECIMAL(12,2);

    SELECT
        @x = ISNULL(SUM(pl.Iznos), 0)
    FROM prodaja.Rezervacije r
    INNER JOIN prodaja.Placanja pl
        ON r.RezervacijaID = pl.RezervacijaID
    WHERE
        r.ProjekcijaID = @ProjekcijaID
        AND pl.StatusPlacanja = N'Placeno';

    RETURN ISNULL(@x, 0);
END;
GO
```

Slika 36 - UDF za izračunavanje prihoda projekcije

9. KREIRANJE USKLADIŠTENIH PROCEDURA ZA AUTOMATIZACIJU

Uskladištene procedure su prethodno kompajlirani SQL kod koji se izvršava na serveru. One omogućavaju ponovno korištenje koda, bolje performanse jer se SQL kod kompajlira jednom prilikom kreiranja procedure, a zatim se izvršava više puta bez ponovnog kompajliranja, te povećanu sigurnost jer se korisnicima može dati dozvola za izvršavanje procedure bez davanja direktnog pristupa tabelama.

9.1. Procedura sp_KreirajRezervaciju

Prva procedura, nazvana sp_KreirajRezervaciju, automatizuje proces kreiranja nove rezervacije. Ova procedura prihvata parametre koji definišu rezervaciju, provjerava ispravnost podataka, kreira rezervaciju i ažurira broj posjeta kupca. Cijela operacija je obavljena transakcijom, što znači da će se ili sve promjene primijeniti ili nijedna, u slučaju greške.

Procedura prihvata sljedeće parametre: KupacID, ProjekcijaID i SjedišteID. Unutar procedure, prvo se provjerava da li kupac postoji u bazi. Ako kupac ne postoji, procedura prekida izvršavanje sa odgovarajućom porukom o grešci. Zatim se provjerava da li je sjedište slobodno za datu projekciju korištenjem funkcije fn_SlobodnaSjedista. Ako sjedište nije slobodno, procedura također prekida izvršavanje. Nakon toga, procedura provjerava da li je projekcija dostupna, odnosno da li ima status Planirana. Ako projekcija nije dostupna, procedura prekida izvršavanje.

Nakon što su sve provjere prošle, procedura dohvata osnovnu cijenu projekcije i broj posjeta kupca. Zatim izračunava popust korištenjem funkcije fn_PopustKupca i primjenjuje ga na cijenu karte. Procedura unosi novu rezervaciju u tabelu Rezervacije sa statusom Potvrđena i rokom potvrde postavljenim na trideset minuta od trenutka kreiranja. Zatim unosi rezervisano sjedište u tabelu RezervisanaSjedista sa izračunatom cijenom. Na kraju, ažurira broj posjeta kupca tako što ga povećava za jedan. Ako sve naredbe izvrše uspješno, transakcija se potvrđuje i procedura vraća identifikator novokreirane rezervacije. U slučaju bilo kakve greške, transakcija se poništava i greška se prosleđuje pozivaocu procedure.

9.2. Procedura sp_OtkaziRezervaciju

Druga procedura, nazvana sp_OtkaziRezervaciju, automatizuje proces otkazivanja rezervacije. Ova procedura prihvata identifikator rezervacije kao parametar i ažurira status rezervacije u Otkazana. Također, ažurira status svih plaćanja vezanih za tu rezervaciju u Refundirano, što omogućava praćenje povrata novca.

```
CREATE PROCEDURE prodaja.sp_OtkaziRezervaciju
    @RezervacijaID INT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE prodaja.Rezervacije
    SET StatusRezervacije = N'Otkazana'
    WHERE RezervacijaID = @RezervacijaID;

    UPDATE prodaja.Placanja
    SET StatusPlacanja = N'Refundirano'
    WHERE RezervacijaID = @RezervacijaID
        AND StatusPlacanja = N'Placeno';
END;
GO
```

Slika 37 - Procedura za otkazivanje rezervacije

Ova procedura je jednostavna, ali važna za održavanje konzistentnosti podataka. Otkazivanjem rezervacije, sistem automatski označava sva povezana plaćanja kao refundirana, što je važno za finansijsko izvještavanje.

9.3. Procedura sp_ProjekcijeZaPeriod

Treća procedura, nazvana sp_ProjekcijeZaPeriod, kreirana je za potrebe izvještavanja o projekcijama u određenom vremenskom periodu. Ova procedura prihvata početni i krajnji datum kao parametre i vraća sve projekcije koje se nalaze u tom periodu, koristeći pogled vw_RasporedProjekcija. Ova procedura je korisna za operativni rad jer omogućava brz pregled rasporeda za bilo koji vremenski interval.

9.4. Procedura sp_PrihodPeriod

Četvrta procedura, nazvana sp_PrihodPeriod, kreirana je za potrebe finansijskog izvještavanja o prihodima u određenom vremenskom periodu. Ova procedura prihvata početni i krajnji datum kao parametre i vraća prihod po filmovima za taj period.

Procedura spaja podatke iz tabela Placanja, Rezervacije, Projekcije i Filmovi, grupira ih po filmovima i prikazuje broj uplata, ukupan prihod i prosječnu uplatu. Rezultati su sortirani po prihodu opadajuće, što omogućava brz uvid u najprofitabilnije filmove.

10. KREIRANJE TRIGERA ZA AUTOMATSKO IZVRŠAVANJE RADNJI

Trigeri su posebne vrste uskladištenih procedura koje se ne pozivaju eksplicitno, već se automatski izvršavaju kada se desi određeni događaj na tabeli, kao što su INSERT, UPDATE ili DELETE operacije.

10.1. Tabela za praćenje promjena rezervacija

Prije kreiranja trigera za praćenje promjena rezervacija, potrebno je kreirati tabelu koja će čuvati istoriju promjena. Ova tabela, nazvana RezervacijeDnevnik, sadrži identifikator dnevnika, identifikator rezervacije, vrstu akcije, stari status, novi status, datum akcije i korisnika koji je izvršio promjenu. SUSER_SNAME() funkcija automatski bilježi ime korisnika koji je izvršio promjenu.

```
CREATE TABLE audit.RezervacijeDnevnik (  
    DnevnikID INT IDENTITY PRIMARY KEY,  
    RezervacijaID INT NOT NULL,  
    Akcija NVARCHAR(10) NOT NULL,  
    StariStatus NVARCHAR(20),  
    NoviStatus NVARCHAR(20),  
    DatumAkcije DATETIME NOT NULL DEFAULT GETDATE(),  
    Korisnik NVARCHAR(128) NOT NULL DEFAULT SUSER_SNAME()  
);
```

Slika 38 - Tabela za praćenje promjena rezervacija

10.2. Triger trg_Rezervacije_Dnevnik

Prvi trigger, nazvan trg_Rezervacije_Dnevnik, aktivira se nakon INSERT, UPDATE ili DELETE operacije nad tabelom Rezervacije. Njegova svrha je da evidentira svaku promjenu nad rezervacijama u dnevnik, omogućavajući kasniju reviziju i praćenje istorije.

```
CREATE TRIGGER prodaja.trg_Rezervacije_Dnevnik  
ON prodaja.Rezervacije  
AFTER INSERT, UPDATE, DELETE  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    INSERT INTO audit.RezervacijeDnevnik  
    (  
        RezervacijaID,  
        Akcija,  
        StariStatus,  
        NoviStatus  
    )  
    SELECT  
        ISNULL(i.RezervacijaID, d.RezervacijaID),  
        CASE  
            WHEN d.RezervacijaID IS NULL THEN N'INSERT'  
            WHEN i.RezervacijaID IS NULL THEN N'DELETE'  
            ELSE N'UPDATE'  
        END,  
        d.StatusRezervacije,  
        i.StatusRezervacije  
    FROM inserted i  
    FULL OUTER JOIN deleted d  
        ON i.RezervacijaID = d.RezervacijaID;  
END;  
GO
```

Slika 39 - Query za kreiranje trigera trg_Rezervacije_Dnevnik

Trigger koristi specijalne tabele inserted i deleted koje SQL Server automatski kreira u kontekstu triggera. Tabela inserted sadrži nove vrijednosti nakon INSERT ili UPDATE operacije, dok tabela deleted sadrži stare vrijednosti prije DELETE ili UPDATE operacije. Trigger pomoću FULL OUTER JOIN-a kombinuje ove dvije tabele kako bi odredio vrstu akcije. Ako je inserted NULL, radi se o DELETE operaciji. Ako je deleted NULL, radi se o INSERT operaciji. Ako obje tabele imaju vrijednosti, radi se o UPDATE operaciji. Za svaku promjenu, trigger upisuje zapis u dnevnik sa odgovarajućim podacima.

10.3. Tabela za praćenje promjena cijena projekcija

Također je potrebno kreirati tabelu za praćenje promjena cijena projekcija. Ova tabela, nazvana CijeneProjekcijaDnevnik, sadrži identifikator dnevnika, identifikator projekcije, staru cijenu, novu cijenu, datum promjene i korisnika koji je izvršio promjenu.

```
CREATE TABLE audit.CijeneProjekcijaDnevnik (  
    DnevnikID INT IDENTITY PRIMARY KEY,  
    ProjekcijaID INT NOT NULL,  
    StaraCijena DECIMAL(10,2),  
    NovaCijena DECIMAL(10,2),  
    DatumPromjene DATETIME NOT NULL DEFAULT GETDATE(),  
    Korisnik NVARCHAR(128) NOT NULL DEFAULT SUSER_SNAME()  
);
```

Slika 40 - Tabela za praćenje promjena cijena projekcija

10.4. Triger trg_Projekcije_Cijena

Drugi trigger, nazvan trg_Projekcije_Cijena, aktivira se nakon UPDATE operacije nad tabelom Projekcije. Njegova svrha je da evidentira svaku promjenu cijene projekcije u log tabelu. Trigger prvo provjerava da li je UPDATE operacija uticala na kolonu OsnovnaCijena. Zatim upisuje zapis u log tabelu sa starom cijenom i novom cijenom za svaku projekciju čija je cijena promijenjena.

```

CREATE TRIGGER kino.trg_Projekcije_Cijena
ON kino.Projekcije
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO audit.CijeneProjekcijaDnevnik
    (
        ProjekcijaID,
        StaraCijena,
        NovaCijena
    )
    SELECT
        i.ProjekcijaID,
        d.OsnovnaCijena,
        i.OsnovnaCijena
    FROM inserted i
    INNER JOIN deleted d
        ON i.ProjekcijaID = d.ProjekcijaID
    WHERE i.OsnovnaCijena <> d.OsnovnaCijena;
END;
GO

```

Slika 41 - Query za kreiranje triger trg_Projekcije_Cijena

10.5. Triger trg_RS_ProvjeraSale

Treći trigger, nazvan trg_RS_ProvjeraSale, aktivira se nakon INSERT ili UPDATE operacije nad tabelom RezervisanaSjedista. Njegova svrha je da provjeri da li sjedište koje se rezerviše pripada istoj sali u kojoj se održava projekcija. Ovo je važno jer sprečava greške gdje bi se rezervisalo sjedište iz pogrešne sale.

```

CREATE TRIGGER prodaja.trg_RS_ProvjeraSale
ON prodaja.RezervisanaSjedista
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS
    (
        SELECT 1
        FROM inserted i
        INNER JOIN prodaja.Rezervacije r
            ON i.RezervacijaID = r.RezervacijaID
        INNER JOIN kino.Projekcije p
            ON r.ProjekcijaID = p.ProjekcijaID
        INNER JOIN kino.Sjedista s
            ON i.SjedisteID = s.SjedisteID
        WHERE p.SalaID <> s.SalaID
    )
    BEGIN
        RAISERROR(N'Sjediste ne pripada sali projekcije.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
GO

```

Slika 42 - Query za kreiranje triger trg_RS_ProvjeraSale

11. PRIMJERI DML NAREDBI ZA MANIPULACIJU PODACIMA

DML naredbe se koriste za manipulaciju podacima u bazi. To su osnovne operacije koje svaki korisnički interfejs izvršava nad bazom podataka. U ovom poglavlju prikazani su konkretni primjeri svih DML naredbi nad bazom Bioskop.

11.1. SELECT naredba za dohvaćanje podataka

Prvi primjer prikazuje dohvaćanje svih kupaca sa konvertovanim imenima u velika slova, prezimenima u mala slova, dužinom imena i prezimena, prva tri slova prezimena, brojem dana od registracije i zamjenom NULL telefona sa tekstom Nema.

```
SELECT KupacID, UPPER(Ime), LOWER(Prezime), LEN(Ime+Prezime),  
LEFT(Prezime,3), DATEDIFF(DAY,DatumRegistracije,GETDATE()),  
ISNULL(Telefon,N'Nema') FROM prodaja.Kupci
```

Drugi primjer prikazuje statističke podatke o filmovima, uključujući ukupan broj filmova, minimalno trajanje, maksimalno trajanje i prosječno trajanje.

```
SELECT COUNT(*) BrojFilmova, MIN(TrajanjeMin) Najkraci, MAX(TrajanjeMin)  
Najduzi, AVG(CAST(TrajanjeMin AS DECIMAL(10,2))) Prosjek FROM kino.Filmovi
```

Treći primjer prikazuje projekcije sa originalnom cijenom, cijenom sa porezom od 17 posto, zaokruženom cijenom na veći cijeli broj, zaokruženom cijenom na manji cijeli broj i kategorijom cijene.

```
SELECT ProjekcijaID, OsnovnaCijena, ROUND(OsnovnaCijena*1.17,2),  
CEILING(OsnovnaCijena), FLOOR(OsnovnaCijena), CASE WHEN OsnovnaCijena<10  
THEN N'Povoljna' WHEN OsnovnaCijena<=15 THEN N'Srednja' ELSE N'Premium'  
END Kategorija FROM kino.Projekcije
```

11.2. INSERT naredba za unos novih podataka

Primjer INSERT naredbe prikazuje unos novog kupca u tabelu Kupci. Ova naredba se koristi za registraciju novih korisnika u sistem.

```
INSERT INTO prodaja.Kupci(Ime,Prezime,Email,Telefon,BrojPosjeta)  
VALUES(N'Test',N'Kupac',N'test.kupac@primjer.com',N'065999999',0)
```

11.3. UPDATE naredba za izmjenu postojećih podataka

Primjer UPDATE naredbe prikazuje otkazivanje rezervacije promjenom njenog statusa. Ova naredba se koristi za ažuriranje postojećih podataka u tabelama.

```
UPDATE prodaja.Rezervacije SET StatusRezervacije=N'Otkazana' WHERE  
RezervacijaID=1
```

11.4. DELETE naredba za brisanje podataka

Primjer DELETE naredbe prikazuje brisanje rezervacije. Ova naredba se koristi za uklanjanje podataka iz tabela. U produkcijskom okruženju, brisanje podataka se obično izbjegava u korist logičkog brisanja putem statusa.

```
DELETE FROM prodaja.Rezervacije WHERE RezervacijaID=1
```

12. PRIMJERI DDL NAREDBI ZA DEFINISANJE STRUKTURE

DDL naredbe se koriste za definisanje i izmjenu strukture baze podataka. Za razliku od DML naredbi koje manipulišu podacima, DDL naredbe definišu objekte u bazi.

12.1. ALTER naredba za izmjenu strukture

ALTER naredba se koristi za izmjenu postojećih objekata u bazi podataka. U nastavku je prikazan primjer dodavanja nove kolone u tabelu Kupci. Ova kolona će čuvati informaciju o tome da li kupac želi da prima obavijesti.

ALTER TABLE prodaja.Kupci ADD PrimaObavijesti BIT NOT NULL DEFAULT 1

12.2. DROP naredba za brisanje objekata

DROP naredba se koristi za brisanje objekata iz baze podataka. Ovo treba raditi oprezno jer se podaci trajno gube. Primjer prikazuje brisanje indeksa koji više nije potreban.

DROP INDEX IX_Sjedista_Sala ON kino.Sjedista

12.3. CREATE naredba za kreiranje objekata

CREATE naredba se koristi za kreiranje novih objekata u bazi podataka. Primjer prikazuje kreiranje nove tabele za čuvanje dnevnih prihoda.

12.4. TRUNCATE naredba za brzo brisanje svih podataka

TRUNCATE je DDL naredba koja briše sve redove iz tabele, ali zadržava strukturu tabele. Brža je od DELETE jer ne zapisuje svaki red pojedinačno. Primjer prikazuje brisanje svih zapisa iz tabele DnevniPrihod.

TRUNCATE TABLE izvjestaji.DnevniPrihod

Razlika između DELETE i TRUNCATE je značajna. DELETE je sporija jer zapisuje svaki red i može koristiti WHERE uslov, aktivira trigere i ne resetuje IDENTITY. TRUNCATE je veoma brza, ne može koristiti WHERE uslov, ne aktivira trigere i resetuje IDENTITY na početnu vrijednost.

13. PRIMJERI KORIŠTENJA RAZLIČITIH TIPOVA FUNKCIJA

SQL Server nudi bogat skup ugrađenih funkcija koje se mogu koristiti u upitima. U ovom poglavlju prikazani su primjeri korištenja različitih tipova funkcija nad bazom Bioskop.

13.1. Datumske funkcije

Datumske funkcije se koriste za rad sa datumima i vremenom. U primjeru ispod, za svakog kupca se prikazuje datum registracije i broj dana od registracije do danas, korištenjem DATEDIFF funkcije sa DAY parametrom.

```
SELECT Ime, Prezime, DatumRegistracije, DATEDIFF(DAY, DatumRegistracije,  
GETDATE()) AS DaniOdRegistracije FROM prodaja.Kupci
```

13.2. String funkcije

String funkcije se koriste za manipulaciju tekstualnim podacima. U primjeru ispod, prikazana je konverzija imena u velika slova, prezimena u mala slova, izračunavanje dužine kombinacije imena i prezimena, te izdvajanje prva tri slova prezimena.

```
SELECT UPPER(Ime) AS ImeVeliko, LOWER(Prezime) AS PrezimeMalo, LEN(Ime + ' '  
+ Prezime) AS DuzinaImena, LEFT(Prezime, 3) AS PrvaTriSlova FROM prodaja.Kupci
```

13.3. Agregatne funkcije

Agregatne funkcije se koriste za izračunavanje statističkih vrijednosti nad skupovima podataka. U primjeru ispod, izračunava se ukupan broj filmova, minimalno trajanje, maksimalno trajanje i prosječno trajanje filmova.

```
SELECT COUNT(*) AS BrojFilmova, MIN(TrajanjeMin) AS NajkraciFilm,  
MAX(TrajanjeMin) AS NajduziFilm, AVG(CAST(TrajanjeMin AS DECIMAL(10,2))) AS  
ProsjecnoTrajanje FROM kino.Filmovi
```

13.4. Numeričke funkcije

Numeričke funkcije se koriste za matematičke operacije nad numeričkim podacima. U primjeru ispod, prikazane su različite numeričke operacije nad cijenama projekcija, uključujući zaokruživanje, cijenu sa porezom i kategorizaciju cijena.

```
SELECT ProjekcijaID, OsnovnaCijena, ROUND(OsnovnaCijena * 1.17, 2) AS  
CijenaSaPorezom, CEILING(OsnovnaCijena) AS ZaokruženoNavise,  
FLOOR(OsnovnaCijena) AS ZaokruženoNanize FROM kino.Projekcije
```

14. PRIMJERI JOIN UPITA ZA KOMBINOVANJE PODATAKA IZ VIŠE TABELA

JOIN operacije se koriste za kombinovanje podataka iz dvije ili više tabela na osnovu veze između njih. U relacionim bazama podataka, podaci su često raspoređeni po različitim tabelama, a JOIN operacije omogućavaju njihovo povezivanje u smislene cijeline.

14.1. INNER JOIN

INNER JOIN vraća samo one redove koji imaju podudaranje u obe tabele koje se spajaju. Ovo je najčešće korištena vrsta JOIN-a. U primjeru ispod, prikazani su svi filmovi sa njihovim projekcijama i odgovarajućim salama.

```
SELECT f.Naziv, p.DatumVrijeme, s.Naziv AS Sala FROM kino.Projekcije p INNER  
JOIN kino.Filmovi f ON p.FilmID = f.FilmID INNER JOIN kino.Sale s ON p.SalaID =  
s.SalaID
```

14.2. LEFT JOIN

LEFT JOIN vraća sve redove iz lijeve tabele, a samo podudarajuće iz desne tabele. Ako nema podudaranja, kolone iz desne tabele će biti NULL. U primjeru ispod, prikazani su svi kupci, bez obzira da li imaju rezervacije ili ne.

```
SELECT k.KupacID, k.Ime, k.Prezime, COUNT(r.RezervacijaID) AS BrojRezervacija  
FROM prodaja.Kupci k LEFT JOIN prodaja.Rezervacije r ON k.KupacID = r.KupacID  
GROUP BY k.KupacID, k.Ime, k.Prezime
```

14.3. RIGHT JOIN

RIGHT JOIN vraća sve redove iz desne tabele, a samo podudarajuće iz lijeve tabele. Ovo je simetrično LEFT JOIN-u. U primjeru ispod, prikazane su sve projekcije sa pripadajućim rezervacijama, uključujući i one projekcije koje nemaju rezervacije.

```
SELECT p.ProjekcijaID, r.RezervacijaID FROM prodaja.Rezervacije r RIGHT JOIN  
kino.Projekcije p ON r.ProjekcijaID = p.ProjekcijaID
```

14.4. FULL OUTER JOIN

FULL OUTER JOIN vraća sve redove iz obje tabele, spajajući gdje postoji podudaranje. Tamo gdje nema podudaranja, kolone iz suprotne tabele će biti NULL. U primjeru ispod, prikazane su sve kombinacije kupaca i rezervacija.

```
SELECT k.Ime, r.RezervacijaID FROM prodaja.Kupci k FULL OUTER JOIN  
prodaja.Rezervacije r ON k.KupacID = r.KupacID
```

14.5. CROSS JOIN

CROSS JOIN vraća kartezijev proizvod dvije tabele, što znači da se svaki red iz prve tabele spaja sa svakim redom iz druge tabele. Ovo se rijetko koristi u praksi jer može proizvesti ogroman broj redova. U primjeru ispod, prikazane su sve moguće kombinacije filmova i sala.

```
SELECT f.Naziv, s.Naziv FROM kino.Filmovi f CROSS JOIN kino.Sale s
```

14.6. SELF JOIN

SELF JOIN je spajanje tabele same sa sobom. Koristi se kada treba uporediti redove unutar iste tabele. U primjeru ispod, prikazani su parovi filmova koji pripadaju istom žanru.

```
SELECT f1.Naziv AS Film1, f2.Naziv AS Film2, f1.ZanrID FROM kino.Filmovi f1 INNER  
JOIN kino.Filmovi f2 ON f1.ZanrID = f2.ZanrID AND f1.FilmID < f2.FilmID
```

15. PRIMJERI PODUPITA I SKUPOVNIH OPERATORA

Podupiti su upiti ugniježđeni unutar drugih upita, dok skupovni operatori omogućavaju kombinovanje rezultata više upita.

15.1. Podupiti

Prvi primjer podupita prikazuje filmove čije je trajanje veće od prosječnog trajanja svih filmova. Podupit izračunava prosječno trajanje, a spoljašnji upit filtrira filmove koji su duži od tog prosjeka.

```
SELECT Naziv, TrajanjeMin FROM kino.Filmovi WHERE TrajanjeMin > (SELECT  
AVG(TrajanjeMin) FROM kino.Filmovi)
```

Drugi primjer podupita prikazuje projekcije koje nemaju nijednu rezervaciju. Koristi se NOT EXISTS operator koji provjerava da li postoji rezervacija za datu projekciju.

```
SELECT p.ProjekcijaID, p.DatumVrijeme FROM kino.Projekcije p WHERE NOT EXISTS  
(SELECT 1 FROM prodaja.Rezervacije r WHERE r.ProjekcijaID = p.ProjekcijaID)
```

Treći primjer podupita prikazuje kupce koji imaju broj posjeta veći od prosječnog broja posjeta svih kupaca.

```
SELECT Ime, Prezime, BrojPosjeta FROM prodaja.Kupci WHERE BrojPosjeta > (SELECT  
AVG(CAST(BrojPosjeta AS DECIMAL(10,2))) FROM prodaja.Kupci)
```

15.2. Skupovni operatori

UNION operator kombinuje rezultate dva upita i uklanja duple redove. Primjer prikazuje sve kupce i zaposlene u jedinstvenoj listi.

```
SELECT Ime, Prezime, N'Kupac' AS Uloga FROM prodaja.Kupci UNION SELECT Ime,  
Prezime, N'Zaposleni' AS Uloga FROM kino.Zaposleni
```

UNION ALL operator kombinuje rezultate dva upita i zadržava sve redove, uključujući duple. Primjer prikazuje sve filmove i sale.

```
SELECT Naziv, N'Film' AS Tip FROM kino.Filmovi UNION ALL SELECT Naziv, N'Sala' AS  
Tip FROM kino.Sale
```

INTERSECT operator vraća redove koji se pojavljuju u oba upita. Primjer prikazuje kupce koji imaju barem jednu rezervaciju.

```
SELECT KupacID FROM prodaja.Rezervacije INTERSECT SELECT KupacID FROM  
prodaja.Kupci
```

EXCEPT operator vraća redove iz prvog upita koji se ne pojavljuju u drugom upitu. Primjer prikazuje kupce koji nemaju nijednu rezervaciju.

```
SELECT KupacID FROM prodaja.Kupci EXCEPT SELECT KupacID FROM  
prodaja.Rezervacije
```

16. KORISNICI, ULOGE I BEZBJEDNOST

Bezbjednost baze podataka je ključni aspekt svakog informacionog sistema. U SQL Serveru, bezbjednost se ostvaruje kroz korisnike, uloge i dodjelu prava. Uloge omogućavaju grupisanje korisnika sa sličnim potrebama za pristupom podacima.

16.1. Kreiranje uloga

Prvi korak u uspostavljanju bezbjednosti jeste kreiranje uloga koje definišu različite nivoe pristupa. U ovom sistemu kreirane su dvije uloge. Uloga `db_bioskop_operater` namijenjena je operativnom osoblju koje radi sa prodajnim podacima. Uloga `db_bioskop_izvjestaji` namijenjena je analitičarima koji pripremaju izvještaje.

```
CREATE ROLE db_bioskop_operater  
CREATE ROLE db_bioskop_izvjestaji
```

16.2. Dodjela prava ulogama

Nakon kreiranja uloga, potrebno je dodijeliti im odgovarajuća prava. Operaterima se daje pravo SELECT, INSERT i UPDATE na shemi prodaja, ali im se uskraćuje pravo DELETE na istoj shemi. Također, operaterima se daje pravo SELECT na shemi kino kako bi mogli vidjeti podatke o filmovima i projekcijama. Analitičarima se daje pravo SELECT i EXECUTE na shemi izvjestaji, što im omogućava da čitaju izvještajne poglede i izvršavaju izvještajne procedure.

```
GRANT SELECT, INSERT, UPDATE ON SCHEMA::prodaja TO db_bioskop_operater  
DENY DELETE ON SCHEMA::prodaja TO db_bioskop_operater  
GRANT SELECT ON SCHEMA::kino TO db_bioskop_operater  
GRANT SELECT, EXECUTE ON SCHEMA::izvjestaji TO db_bioskop_izvjestaji
```

16.3. Kreiranje korisnika

Korisnici se kreiraju i dodjeljuju odgovarajućim ulogama. U ovom primjeru kreirani su korisnici bez SQL Server login-a, što je pogodno za demonstraciju unutar baze podataka. Korisnik BioskopOperater se dodjeljuje ulozi db_bioskop_operater, dok se korisnik BioskopAnaliticar dodjeljuje ulozi db_bioskop_izvjestaji.

```
CREATE USER BioskopOperater WITHOUT LOGIN
```

```
CREATE USER BioskopAnaliticar WITHOUT LOGIN
```

```
EXEC sp_addrolemember N'db_bioskop_operater', N'BioskopOperater'
```

```
EXEC sp_addrolemember N'db_bioskop_izvjestaji', N'BioskopAnaliticar'
```

17. ETL PROCESI

ETL procesi se koriste za ekstrakciju podataka iz izvornih sistema, njihovu transformaciju i učitavanje u ciljnu bazu podataka. U ovom sistemu implementirana su dva ETL procesa.

17.1. Tabela StageFilmovi

Prvi korak u ETL procesu je kreiranje staging tabele koja prima podatke iz izvornog sistema. Tabela StageFilmovi sadrži podatke o filmovima koji trebaju biti uvezeni u glavni sistem. Kolona Obradjen se koristi za praćenje koji redovi su već obrađeni.

```
CREATE TABLE etl.StageFilmovi
(
    IzvorniID INT,
    Naziv NVARCHAR(150),
    Zavr NVARCHAR(50),
    Kategorija NVARCHAR(50),
    TrajanjeMin INT,
    Godina SMALLINT,
    Obradjen BIT NOT NULL DEFAULT 0
);
```

Slika 43 - Tabela StageFilmovi

U tabelu se unose testni podaci koji predstavljaju filmove koji čekaju na uvoz.

```
INSERT INTO etl.StageFilmovi VALUES(1001, N' Povratak kući ', N'Drama', N'12+', 109,
2026, 0)
```

```
INSERT INTO etl.StageFilmovi VALUES(1002, N'Mali hero', N'Animirani', N'Svi uzrasti', 87,
2026, 0)
```

17.2. Procedura sp_UveziFilmove

Prvi ETL proces je implementiran kroz proceduru sp_UveziFilmove. Ova procedura učitava podatke iz staging tabele u glavnu tabelu Filmovi. Prije uvoza, podaci se čiste korištenjem LTRIM i RTRIM funkcija za uklanjanje suvišnih razmaka. Podaci se povezuju sa šifarnicima žanrova i kategorija kako bi se dobili odgovarajući identifikatori.

```

CREATE PROCEDURE etl.sp_UveziFilmove
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRAN;

    INSERT INTO kino.Filmovi
    (
        ZanrID,
        KategorijaID,
        Naziv,
        TrajanjeMin,
        Godina
    )
    SELECT
        z.ZanrID,
        k.KategorijaID,
        LTRIM(RTRIM(s.Naziv)),
        s.TrajanjeMin,
        s.Godina
    FROM etl.StageFilmovi s
    INNER JOIN sif.Zanrovi z
        ON s.Zanr = z.Naziv
    INNER JOIN sif.Kategorije k
        ON s.Kategorija = k.Naziv
    WHERE
        s.Obradjen = 0
        AND NOT EXISTS
        (
            SELECT 1
            FROM kino.Filmovi f
            WHERE f.Naziv = LTRIM(RTRIM(s.Naziv))
                AND f.Godina = s.Godina
        );

    UPDATE etl.StageFilmovi
    SET Obradjen = 1
    WHERE Obradjen = 0;

    COMMIT;
END;
GO

```

Slika 44 - Procedura za uvoz filmova

Procedura provjerava da li film već postoji u glavnoj tabeli prije uvoza, kako bi se spriječilo dupliranje. Nakon uspješnog uvoza, redovi u staging tabeli se označavaju kao obrađeni. Cijeli proces je obavljen transakcijom.

17.3. Tabela DnevniPrihod

Drugi ETL proces zahtijeva kreiranje izvještajne tabele koja će čuvati dnevne prihode po filmovima. Tabela DnevniPrihod sadrži datum, identifikator filma, broj karata, prihod i datum učitavanja.

```
CREATE TABLE izvjestaji.DnevniPrihod
(
    Datum DATE NOT NULL,
    FilmID INT NOT NULL,
    BrojKarata INT NOT NULL,
    Prihod DECIMAL(12,2) NOT NULL,
    DatumUcitavanja DATETIME NOT NULL DEFAULT GETDATE(),
    PRIMARY KEY(Datum, FilmID)
);
GO
```

Slika 45 - Tabela DnevniPrihod

17.4. Procedura sp_UcitajDnevniPrihod

Drugi ETL proces je implementiran kroz proceduru sp_UcitajDnevniPrihod. Ova procedura izračunava dnevni prihod za određeni datum i učitava ga u izvještajnu tabelu. Procedura prvo briše sve postojeće zapise za traženi datum, a zatim izračunava broj karata i prihod po filmovima za taj datum. Podaci se dovode iz tabela Filmovi, Projekcije, Rezervacije i Placanja, sa filtriranjem na datum plaćanja i status placeno. Rezultat se učitava u izvještajnu tabelu.

18. KREIRANJE REZERVNE KOPIJE BAZE PODATAKA

Redovno kreiranje rezervnih kopija predstavlja jednu od najvažnijih aktivnosti u administraciji baza podataka. Backup omogućava oporavak sistema u slučaju gubitka podataka usljed greške korisnika, kvara hardvera, softverskih problema ili sigurnosnih incidenata. Sistem za upravljanje bazama podataka Microsoft SQL Server omogućava više vrsta backup-a koje se koriste u zavisnosti od potreba sistema i učestalosti promjena podataka.

18.1. Kreiranje potpunog backup-a

Potpuni backup kreira kompletnu kopiju cijele baze podataka, uključujući sve tabele, indekse, procedure i podatke. Ovo je osnovni tip backup-a i predstavlja polaznu tačku za sve ostale vrste backup-a. U naredbi BACKUP DATABASE navodi se ime baze koju želimo da backup-ujemo, dok se pomoću opcije TO DISK definiše lokacija na kojoj će backup fajl biti sačuvan. Opcije koje se koriste uključuju INIT za prepisivanje postojećeg sadržaja, CHECKSUM za provjeru integriteta, COMPRESSION za kompresiju backup fajla, NAME za naziv backup seta i STATS za prikaz napretka u procentima.

```
BACKUP DATABASE Bioskop TO DISK=N'C:\SQLBackup\Bioskop_FULL.bak' WITH INIT,  
CHECKSUM, COMPRESSION, NAME=N'Full backup', STATS=10
```

18.2. Diferencijalni backup

Diferencijalni backup čuva samo promjene koje su nastale nakon posljednjeg potpunog backup-a. Ova vrsta backup-a je brža od potpunog i zahtijeva manje prostora, ali se mora kombinovati sa potpunim backup-om za potpunu restauraciju.

```
BACKUP DATABASE Bioskop TO DISK=N'C:\SQLBackup\Bioskop_DIFF.bak' WITH  
DIFFERENTIAL, INIT, CHECKSUM, STATS=10
```

18.3. Backup loga transakcija

Ova vrsta backup-a čuva sve transakcije koje su izvršene nad bazom. Omogućava vraćanje baze na tačno određeni trenutak. Da bi se koristio backup loga, baza mora biti u FULL recovery modelu.

```
ALTER DATABASE Bioskop SET RECOVERY FULL  
BACKUP DATABASE Bioskop TO  
DISK=N'C:\SQLBackup\Bioskop_FULL_LOGCHAIN.bak' WITH INIT, CHECKSUM  
BACKUP LOG Bioskop TO DISK=N'C:\SQLBackup\Bioskop_LOG.trn' WITH INIT,  
CHECKSUM, STATS=10
```

18.4. Copy-only backup

Copy-only backup se koristi kada želimo napraviti backup bez uticaja na postojeći backup lanac. Ovo je korisno kada treba napraviti dodatnu kopiju bez narušavanja redovnog backup rasporeda.

```
BACKUP DATABASE Bioskop TO DISK=N'C:\SQLBackup\Bioskop_COPYONLY.bak'  
WITH COPY_ONLY, INIT, CHECKSUM
```

18.5. Ostale vrste backup-a

SQL Server podržava i druge vrste backup-a. File backup omogućava backup pojedinačnih datoteka baze. Filegroup backup omogućava backup grupa datoteka. Partial backup kreira kopiju samo primarne filegrupe i čitljivih filegrupa. Striped backup distribuira backup preko više fajlova radi ubrzanja i veće fleksibilnosti. Tail-log backup pravi kopiju posljednjeg dijela loga prije oporavka.

```
BACKUP DATABASE Bioskop FILE=N'Bioskop' TO  
DISK=N'C:\SQLBackup\Bioskop_FILE.bak' WITH INIT, CHECKSUM  
BACKUP DATABASE Bioskop FILEGROUP=N'PRIMARY' TO  
DISK=N'C:\SQLBackup\Bioskop_FILEGROUP.bak' WITH INIT, CHECKSUM  
BACKUP DATABASE Bioskop READ_WRITE_FILEGROUPS TO  
DISK=N'C:\SQLBackup\Bioskop_PARTIAL.bak' WITH INIT, CHECKSUM
```

```
BACKUP DATABASE Bioskop READ_WRITE_FILEGROUPS TO  
DISK=N'C:\SQLBackup\Bioskop_PARTIAL_DIFF.bak' WITH DIFFERENTIAL, INIT,  
CHECKSUM  
BACKUP DATABASE Bioskop TO DISK=N'C:\SQLBackup\Bioskop_STRIPE1.bak',  
DISK=N'C:\SQLBackup\Bioskop_STRIPE2.bak' WITH INIT, CHECKSUM  
BACKUP LOG Bioskop TO DISK=N'C:\SQLBackup\Bioskop_TAIL.trn' WITH  
NORECOVERY, CONTINUE_AFTER_ERROR
```

18.6. Informacije o backup-u

Prije restauracije, korisno je dobiti informacije o backup fajlu. HEADERONLY prikazuje zaglavlje backup seta. FILELISTONLY prikazuje listu datoteka unutar backup-a. VERIFYONLY provjerava integritet backup fajla.

```
RESTORE HEADERONLY FROM DISK=N'C:\SQLBackup\Bioskop_FULL.bak'  
RESTORE FILELISTONLY FROM DISK=N'C:\SQLBackup\Bioskop_FULL.bak'  
RESTORE VERIFYONLY FROM DISK=N'C:\SQLBackup\Bioskop_FULL.bak' WITH  
CHECKSUM
```

18.7. Restauracija baze iz backup-a

U slučaju gubitka podataka ili potrebe za vraćanjem prethodnog stanja, koristi se naredba RESTORE DATABASE. Opcija REPLACE omogućava prepisivanje postojeće baze, dok MOVE omogućava promjenu lokacije datoteka. STATS prikazuje napredak procesa.

```
RESTORE DATABASE Bioskop_Test FROM DISK=N'C:\SQLBackup\Bioskop_FULL.bak'  
WITH MOVE N'Bioskop' TO N'C:\SQLData\Bioskop_Test.mdf, MOVE N'Bioskop_log' TO  
N'C:\SQLData\Bioskop_Test_log.ldf, REPLACE, RECOVERY, STATS=10
```

19. PUBLIKACIJA I TRANSAKCIJSKA REPLIKACIJA

Transakcijska replikacija omogućava distribuciju podataka iz jedne baze u drugu, sa minimalnim kašnjenjem. Ovo je korisno za scenarije gdje je potrebno imati kopiju podataka na udaljenoj lokaciji za izvještavanje ili za smanjenje opterećenja primarne baze.

19.1. Konfiguracija Distributora

Prvi korak u uspostavljanju replikacije je konfiguracija Distributora. Distributor je server koji čuva podatke o replikaciji i distribuira promjene. U ovom primjeru, Distributor je postavljen na isti server kao i Publisher.

USE master

```
EXEC sp_adddistributor @distributor=@@SERVERNAME,  
@password=N'PromijenitiLozinku!'
```

```
EXEC sp_adddistributiondb @database=N'distribution', @data_folder=N'C:\SQLData',  
@log_folder=N'C:\SQLData'
```

```
EXEC sp_adddistpublisher @publisher=@@SERVERNAME, @distribution_db=N'distribution',  
@security_mode=1, @working_directory=N'\VM1\ReplSnapshot'
```

19.2. Kreiranje publikacije

Nakon konfiguracije Distributora, potrebno je kreirati publikaciju koja definiše koje tabele će biti replicirane. Baza podataka se označava kao publikovana, a zatim se kreira publikacija sa određenim svojstvima.

USE Bioskop

```
EXEC sp_replicationdboption @dbname=N'Bioskop', @optname=N'publish', @value=N'true'  
EXEC sp_addpublication @publication=N'Bioskop_Pub', @status=N'active',  
@sync_method=N'concurrent', @repl_freq=N'continuous'
```


19.3. Dodavanje artikala

Artikli definišu koje tabele se publikuju. U ovom primjeru, publikuju se tabele Filmovi i Projekcije iz sheme kino.

```
EXEC sp_addarticle @publication=N'Bioskop_Pub', @article=N'Filmovi',  
@source_owner=N'kino', @source_object=N'Filmovi', @type=N'logbased'  
EXEC sp_addarticle @publication=N'Bioskop_Pub', @article=N'Projekcije',  
@source_owner=N'kino', @source_object=N'Projekcije', @type=N'logbased'
```

19.4. Kreiranje pretplate

Pretplatnik je server koji prima replicirane podatke. U ovom primjeru, pretplatnik je VM2, a podaci se repliciraju u bazu Bioskop_Sub. Push subscription znači da Publisher gura promjene ka Subscriber-u.

```
EXEC sp_addsubscription @publication=N'Bioskop_Pub', @subscriber=N'VM2',  
@destination_db=N'Bioskop_Sub', @subscription_type=N'Push', @sync_type=N'automatic'
```

Nakon uspostavljanja replikacije, promjene koje se izvrše na Publisher serveru automatski se prenose na Subscriber server. Ovo se može testirati unošenjem novog reda na Publisher strani i provjerom da li se isti red pojavljuje na Subscriber strani.

20. ER DIJAGRAM I OPIS RELACIJA

ER dijagram predstavlja grafički prikaz strukture baze podataka, odnosno način na koji su podaci organizovani i međusobno povezani. Pomoću ovog dijagrama moguće je lakše razumjeti kako sistem funkcioniše i kako se podaci kreću kroz aplikaciju. U okviru baze podataka Bioskop definisano je više entiteta koji zajedno čine cjelinu sistema za upravljanje bioskopskim poslovanjem.

20.1. Entiteti i njihovi atributi

Entitet Zanrovi koristi se za čuvanje podataka o žanrovima filmova. Svaki žanr ima svoj jedinstveni identifikator, naziv i opis. Ovaj entitet omogućava kategorizaciju filmova i pretragu po žanru. Entitet Kategorije definiše starosne kategorije filmova. Svaka kategorija ima svoj identifikator, naziv i minimalnu dob koja se zahtijeva za gledanje. Ove kategorije su važne za poštovanje zakonskih regulativa o dobnim ograničenjima. Entitet Filmovi predstavlja centralni entitet za sve informacije o filmovima. Pored osnovnih podataka kao što su naziv i trajanje, evidentiraju se i žanr, kategorija, godina, datum premijere i prosječna ocjena. Atribut Aktivan omogućava da se filmovi koji više nisu u programu isključe iz aktivne ponude. Entitet Sale obuhvata sve projekcije sale u bioskopu. Za svaku salu evidentiraju se naziv, tip, kapacitet, datum otvaranja i status aktivnosti. Različiti tipovi sala omogućavaju različite cjenovne politike. Entitet Sjedišta predstavlja fizički raspored sjedišta u svakoj sali. Svako sjedište ima svoju oznaku reda i broj, te tip koji može biti standardni, VIP ili pristupačan. Ova struktura omogućava preciznu prodaju karata. Entitet Projekcije povezuje filmove, sale i termine. Svaka projekcija ima datum i vrijeme, jezik, format i osnovnu cijenu. Status projekcije omogućava praćenje realizacije rasporeda. Entitet Kupci čuva podatke o registrovanim korisnicima. Svaki kupac ima svoje osnovne podatke, email, telefon, datum registracije i broj posjeta. Ovi podaci omogućavaju personalizaciju i programe lojalnosti. Entitet Rezervacije predstavlja srce prodajnog sistema. Svaka rezervacija povezuje kupca i projekciju, sa datumom rezervacije, statusom i rokom potvrde. Status rezervacije omogućava praćenje njenog životnog ciklusa. Entitet RezervisanaSjedišta omogućava povezivanje rezervacija sa konkretnim sjedištima. Ova tabela rješava vezu više-prema-više između rezervacija i sjedišta, uz evidentiranje cijene pojedinačnog sjedišta. Entitet Placanja služi za evidentiranje finansijskih transakcija. Za svaku uplatu evidentiraju se datum, iznos, način

plaćanja, status i broj transakcije. Ovo omogućava potpunu finansijsku kontrolu. Entitet Cjenovnik definiše cijene karata u zavisnosti od tipa sale i dana u sedmici. Ovo omogućava fleksibilnu cjenovnu politiku. Entitet Zaposleni i entitet Smjene omogućavaju praćenje radnog vremena i organizaciju osoblja.

20.2. Veze između entiteta

Između entiteta postoje jasno definisane relacije koje omogućavaju povezivanje podataka. Jedan žanr može imati više filmova, dok svaki film pripada tačno jednom žanru. Ista relacija postoji između kategorije i filma.

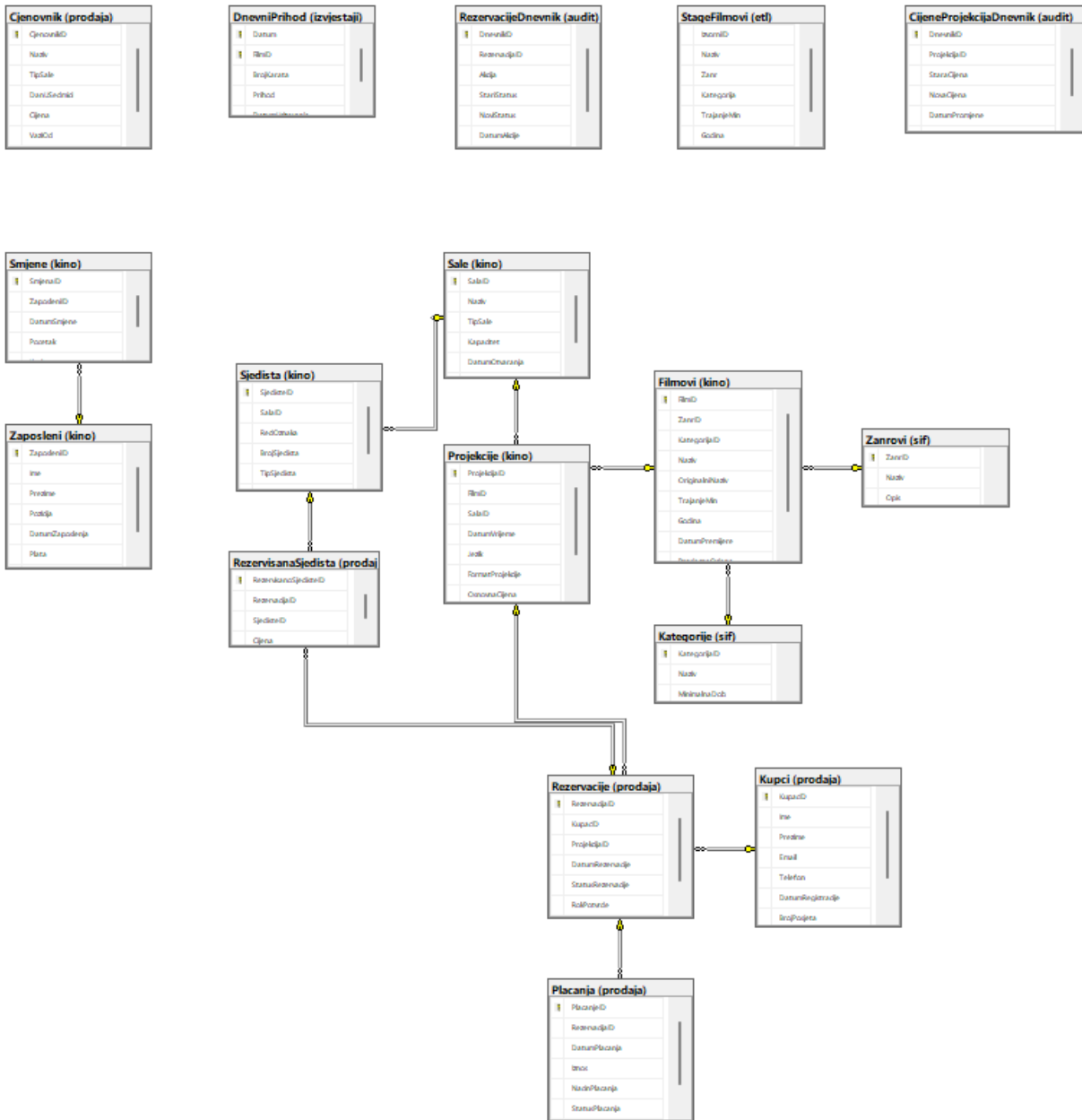
Jedna sala može imati više sjedišta, dok svako sjedište pripada tačno jednoj sali. Jedan film može imati više projekcija, dok svaka projekcija prikazuje tačno jedan film. Jedna sala može biti domaćin više projekcija, dok svaka projekcija se održava u tačno jednoj sali.

Jedan kupac može imati više rezervacija, dok svaka rezervacija pripada tačno jednom kupcu. Jedna projekcija može imati više rezervacija, dok svaka rezervacija se odnosi na tačno jednu projekciju.

Odnos između rezervacija i sjedišta je složeniji jer podrazumijeva vezu više-prema-više. Jedna rezervacija može sadržavati više sjedišta, dok se isto sjedište može pojaviti u više rezervacija kroz različite projekcije. Ovaj odnos je riješen pomoću tabele RezervisanaSjedista.

Jedna rezervacija može imati više plaćanja, dok svako plaćanje pripada tačno jednoj rezervaciji. Jedan zaposleni može imati više smjena, dok svaka smjena pripada tačno jednom zaposlenom.

20.3. Grafički prikaz ER dijagrama



Slika 46 - ER dijagram baze podataka Bioskop

ZAKLJUČAK

U okviru ovog seminarskog rada projektovana je i implementirana baza podataka Bioskop, koja obuhvata osnovne potrebe sistema za upravljanje bioskopskim poslovanjem. Baza se sastoji od trinaest međusobno povezanih tabela, pomoću kojih se vode podaci o žanrovima, kategorijama, filmovima, salama, sjedištima, projekcijama, kupcima, rezervacijama, rezervisanim sjedištima, plaćanjima, cjenovniku, zaposlenima i smjenama.

Primarni i strani ključevi omogućavaju povezivanje tabela i očuvanje referencijalnog integriteta baze. Pored toga, korištena su različita ograničenja koja sprečavaju unos neispravnih ili nelogičnih podataka i na taj način doprinose pouzdanijem radu sistema.

Za poboljšanje performansi korišteni su indeksi, prvenstveno kod upita koji se odnose na filmove, projekcije i rezervacije. Implementirane su i procedure, funkcije i trigeri kojima su automatizovani pojedini procesi, kao što su kreiranje rezervacija, provjera dostupnosti sjedišta i održavanje ispravnosti podataka.

U radu su takođe korišteni pogledi za lakši pregled i prikaz podataka, kao i ETL procesi za unos i obradu podataka. Za zaštitu i kontrolu pristupa bazi definisane su korisničke uloge, dok su backup i replikacija predviđeni kako bi se podaci mogli zaštititi i obnoviti u slučaju problema. Na ovaj način napravljena je funkcionalna baza podataka koja obuhvata glavne procese bioskopskog poslovanja i može predstavljati osnovu za dalji razvoj kompletnog informacionog sistema.

POPIS SLIKA

Slika 1 - Primjer softverskog rješenja za upravljanje bioskopom kompanije CINESync	2
Slika 2 - Kreiranje baze podataka	4
Slika 3 - Aktiviranje baze podataka	4
Slika 4 - Kreiranje shema u bazi podataka.....	5
Slika 5 - Query za kreiranje tabele Zanrovi	6
Slika 6 - Query za kreiranje tabele Kategorije.....	7
Slika 7 - Query za kreiranje tabele Filmovi.....	7
Slika 8 - Query za kreiranje tabele Sale.....	8
Slika 9 - Query za kreiranje tabele Sjedista	9
Slika 10 - Query za kreiranje tabele Projekcije	10
Slika 11 - Query za kreiranje tabele Kupci	11
Slika 12 - Query za kreiranje tabele Rezervacije.....	12
Slika 13 - Query za kreiranje tabele RezervisanaSjedista	12
Slika 14 - Query za kreiranje tabele Placanja	13
Slika 15 - Query za kreiranje tabele Cjenovnik	14
Slika 16 - Query za kreiranje tabele Zaposleni	15
Slika 17 - Query za kreiranje tabele Smjene.....	15
Slika 18 - Unos podataka u tabelu Zanrovi.....	22
Slika 19 - Unos podataka u tabelu Kategorije	22
Slika 20 - Unos podataka u tabelu Sale	23
Slika 21 - Unos podataka u tabelu Filmovi.....	23
Slika 22 - Unos podataka u tabelu Sjedista.....	24
Slika 23 - Unos podataka u tabelu Kupci.....	25
Slika 24 - Unos podataka u tabelu Zaposleni.....	25
Slika 25 - Unos podataka u tabelu Smjene	26
Slika 26 - Unos podataka u tabelu Cjenovnik.....	26
Slika 27 - Unos podataka u tabelu Projekcije	27
Slika 28 - Unos podataka u tabelu Rezervacije	27
Slika 29 - Unos podataka u tabelu RezervisanaSjedista	28
Slika 30 - Unos podataka u tabelu Placanja.....	28
Slika 31 - Query za kreiranje pogleda vw_RasporedProjekcija	29
Slika 32 - Query za kreiranje pogleda vw_RezervacijeDetalji.....	30
Slika 33 - Query za kreiranje pogleda vw_PrihodPoFilmu	31
Slika 34 - UDF za izračunavanje popusta.....	32
Slika 35 - UDF za prikaz slobodnih sjedišta.....	33
Slika 36 - UDF za izračunavanje prihoda projekcije	34
Slika 37 - Procedura za otkazivanje rezervacije	36
Slika 38 - Tabela za praćenje promjena rezervacija	38
Slika 39 - Query za kreiranje triger trg_Rezervacije_Dnevnik.....	38
Slika 40 - Tabela za praćenje promjena cijena projekcija	39
Slika 41 - Query za kreiranje triger trg_Projekcije_Cijena.....	40
Slika 42 - Query za kreiranje triger trg_RS_ProvjeraSale	40
Slika 43 - Tabela StageFilmovi.....	53
Slika 44 - Procedura za uvoz filmova	54

Slika 45 - Tabela DnevniPrihod.....	55
Slika 46 - ER dijagram baze podataka Bioskop.....	63